Department of Computer Science & Engineering
Motilal Nehru National Institute of Technology Allahabad

TECHNICAL PROJECT REPORT: MEDITRACK

A Secure Pharmacy Storefront and Inventory Management System

PROJECT REPORT

Submitted in partial fulfilment of the requirements for
[Secure E-Commerce / Information Security]

Department: CSED

Submitted by:

Abhishek rana

Varshitha

Raj Verma

Guided by: Prof. M. M. Gore

Academic Year: 2026-2027

Document Control

Table 1. Document identification and revision history

Field	Value
Project	MediTrack — secure e-commerce portal for a healthcare store
Document	Technical Project Report (design, implementation and verification)
Revision	2.0 — replaces the draft “MediTrack Project Report v1.0”
Date	30 October 2026
Requirements baseline	MediTrack Software Requirements Specification, Version 1.0 (IEEE 830 structure)
Implementation baseline	Repository `meditrack`, commit `ca672d9` plus the Phase A working-tree changes described in Clause 5 and Clause 7
Reporting standards	IEEE Std 830-1998 (requirements), IEEE Std 1016-2009 (design description), IEEE Std 829-2008 (test documentation), IEEE citation style
Evidence basis	Source-code inspection, automated test execution and production build executed on the development machine (Clause 7.5)

Contents

Document Control i

Table of Contents ii

List of Tables v

List of Figures vi

Abstract vii

Index Terms . . . vii

1 Introduction 1

1.1 Purpose . . . 1
1.2 Scope . . . 1
1.3 Definitions, Acronyms and Abbreviations . . . 1
1.4 Applicable Documents . . . 2
1.5 Report Organisation . . . 2

2 Overall Description 2

2.1 Product Perspective . . . 2
2.2 Product Functions . . . 2
2.3 User Classes and Characteristics . . . 4
2.4 Operating Environment . . . 4
2.5 Design and Implementation Constraints . . . 4
2.6 Assumptions and Dependencies . . . 5

3 Specific Requirements 5

3.1 External Interface Requirements . . . 5
3.2 Functional Requirements . . . 5
3.3 Non-Functional Requirements . . . 7
3.4 Requirements Interpretation and Traceability . . . 8

4 System Design Description 9

4.1 Design Stakeholders and Concerns . . . 9
4.2 Design Views . . . 9
4.2.1 Context view . . . 9
4.2.2 Composition view . . . 11
4.2.3 Logical view . . . 11
4.2.4 Dependency view . . . 12
4.2.5 Information view . . . 13
4.2.6 Interface view . . . 13
4.2.7 Interaction view . . . 14
4.2.8 State dynamics view . . . 15
4.2.9 Algorithm view . . . 15
4.2.10 Resource view . . . 16

4.3 Design Rationale 16

5 Implementation **17**
5.1 Technology Stack 17
5.2 Repository Structure and Size 18
5.3 Backend Modules 18
5.4 Frontend Modules 19
5.5 Security Implementation 20
5.6 Configuration and Tooling 21

6 Data Design and Interfaces **21**
6.1 Data Model 21
6.2 Relationships 22
6.3 API Specification 25
6.4 Error Conventions 26

7 Verification and Validation **26**
7.1 Verification Strategy 26
7.2 Test Environment 27
7.3 Test Suite Inventory 27
7.4 Representative Test Cases 28
7.5 Execution Results 29
7.6 Build and Static Verification 29
7.7 Requirement-to-Test Coverage 29
7.8 Threats to Validity 30

8 Results and Discussion **30**
8.1 Summary of Results 30
8.2 Requirements Alignment 31
8.3 Discussion of Trade-offs 32
8.4 Limitations 32
8.5 Risks and Mitigations 33

9 Conclusion and Future Work **33**
9.1 Conclusion 33
9.2 Future Work 33

References **35**

Appendix A. Requirements Traceability Matrix **36**

Appendix B. API Reference **38**

Appendix C. Configuration Reference **40**

Appendix D. Test Execution Log **41**

Appendix E. Reproduction Procedure **42**

List of Tables

1 Document identification and revision history i
2 Product functions and implementation status 2
3 User classes and their mapping to implemented roles 4
4 Constraints and how they are respected 4
5 External interfaces 5
6 Functional requirements, priority and verified status 6
7 Non-functional requirements and verified status 7
8 Areas where the SRS needs an explicit reading 8
9 Stakeholders, concerns and the views that address them 9
10 Composition view: components and responsibilities 11
11 Layer responsibilities and prohibitions 12
12 Dependency view (selected edges) 12
13 Interface view: access rules per interface 14
14 Interaction view: principal use cases 14
15 Order states, allowed transitions and effects 15
16 Design decisions and rejected alternatives 17
17 Technology stack (versions from the project manifests) 18
18 Repository structure and measured size 18
19 Security controls mapped to requirements 20
20 Collections, key fields and purpose 21
21 Indexes and the invariant each one enforces 22
22 API groups and representative endpoints 25
23 Test levels applied 26
24 Test suites and scope 27
25 Representative test cases with expected and observed results 28
26 Executed test results (this revision) 29
27 Build verification 29
28 Requirement coverage by automated tests 29
29 Measured project indicators 31
30 Requirement-by-requirement assessment 31
31 Risk register 33
32 Future work mapped to requirements 34
33 Traceability from requirement to implementation and evidence 36
34 API surface (part 1: platform, identity and back office) 38
35 Configuration variables 40

List of Figures

1 Context view 10
2 Information flow diagram of the MediTrack portal 10
3 Logical layered view 12
4 Domain class model (UML) of MediTrack 13
5 Checkout and payment interaction 14
6 Order and stock state machine 15
7 ER diagram (part 1 of 3): shopping, cart and wishlist 23
8 ER diagram (part 2 of 3): orders, prescriptions and payments 24
9 ER diagram (part 3 of 3): customer services and security 25

Abstract

MediTrack is a web-based pharmacy storefront combined with a medicine-inventory administration console. It supports catalogue browsing with live stock, guest and signed-in shopping carts, prescription-aware checkout (in progress), sandbox payment, order fulfilment, role-based back-office operations and an append-only audit trail. The system is implemented as a three-tier application: a React single-page frontend, a Node.js/Express REST API and a MongoDB database accessed through Mongoose.

This report documents the project against the MediTrack Software Requirements Specification v1.0, using the requirements, design, implementation and verification clauses prescribed by IEEE Std 830-1998, IEEE Std 1016-2009 and IEEE Std 829-2008. Unlike the earlier draft of this report, which relied on repository documentation alone, every implementation statement here was checked against the source tree, and every quantitative statement was produced by executing the artefact on the development machine.

At the time of writing, the automated backend suite comprises 97 test cases in eight files, all passing, and the frontend production build completes without warnings. Verified capabilities include atomic stock reservation with rollback (FR-2.2, NFR-2.1), server-side pricing computed in paise, a seven-state order lifecycle, card, UPI, net-banking and cash-on-delivery sandbox payments (FR-2.3), low-stock replenishment at a configurable water-mark of 10 units up to 100 (FR-1.1), an idempotent MediPoints ledger credited only after a confirmed payment (FR-2.4), hashed passwords with short-lived access tokens and rotating refresh sessions (NFR-1.2), permission-based authorisation on every protected route (NFR-1.1), and an audit log with no update or delete endpoint (NFR-1.3). Four SRS capabilities remain planned and are specified but not implemented: cart-level soft holds with automatic release (FR-1.2, FR-1.3), a wishlist (FR-1.5), prescription upload and verification gating (FR-1.4, FR-2.1) and refill reminders routed to teleconsultation (FR-2.5). Clause 8.2 gives the item-by-item assessment and Clause 9.2 the plan for closing them.

Index Terms

Pharmacy e-commerce; medicine inventory; stock reservation; concurrency control; MongoDB; Mongoose; React; Express; JSON Web Token; role-based access control; refresh-token rotation; sandbox payment gateway; append-only audit log; IEEE Std 830; IEEE Std 1016; IEEE Std 829.

1 Introduction

1.1 Purpose

This report describes the requirements, design, implementation and verification of MediTrack, a prototype pharmacy storefront. It is intended for the course evaluator, the project guide and the successor team that will continue development, and it serves three purposes: to record what the system does and why it is built that way, to demonstrate traceability from each requirement in the SRS to code and to an executing test, and to state honestly which requirements are not yet met.

1.2 Scope

The reported system covers the customer storefront (catalogue, cart, checkout, payment, order history, MediPoints; wishlist planned), the Inventory Manager console (stock administration, order fulfilment, customers, invitations, audit review, manual replenishment) and the platform services behind them (authentication, sessions, authorisation, pricing, inventory consistency, payment adapter, audit logging). The SRS item "regional demand pattern detection" is explicitly deferred by the SRS itself and is out of scope here.

Payment processing is restricted to a deterministic sandbox provider; no real money moves and no payment credentials are stored. Hardware-dependent barcode/QR and RFID interactions are simulated or deferred (SRS constraints CON-2 and CON-3).

1.3 Definitions, Acronyms and Abbreviations

Term	Definition as used in this report
API	Application Programming Interface; here the REST interface served under `/api`.
Audit log	Append-only collection (`auditlogs`) recording who changed what, before/after values and request metadata.
Cart hold (soft hold)	Short-lived stock reservation created when an item enters a cart (SRS FR-1.2). Planned; see Clause 8.2.
CRM	Customer Relationship Management; purchase history, MediPoints and refill reminders in MediTrack.
JWT	JSON Web Token; the short-lived bearer credential used for API calls.
MediPoints	Loyalty points credited to a customer after a confirmed payment (SRS FR-2.4).
Mongoose	Object-document mapper for MongoDB supplying schemas, validation and index definitions.
NFR	Non-functional requirement.
Paise	One hundredth of a rupee; the integer unit used for all money arithmetic to avoid floating-point drift.
RBAC	Role-Based Access Control; each role maps to explicit permissions in `lib/permissions.js`.
Rx	Prescription. Prescription-only medicines require a verified prescription before billing (SRS FR-1.4, FR-2.1).
Sandbox gateway	Test-mode payment provider with deterministic success/failure rules; default `PAYMENT_PROVIDER=mock`.
TTL	Time To Live; validity window of a token, session, hold or reservation.
SRS	Software Requirements Specification, MediTrack v1.0.

1.4 Applicable Documents

- MediTrack Software Requirements Specification, Version 1.0 (13 pages, prepared per IEEE Std 830-1998 structure) [2].
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications [1].
- IEEE Std 1016-2009, IEEE Standard for Information Technology — Software Design Descriptions [3].
- IEEE Std 829-2008, IEEE Standard for Software and System Test Documentation [4].
- Alba, E., *Writing a Scientific/Technical Report*, Universidad de Málaga, 2002 [5].
- Project source repository and its README [6].

1.5 Report Organisation

Clause 2 gives the overall product description. Clause 3 specifies requirements and their current status. Clause 4 presents the design using the views of IEEE Std 1016. Clause 5 describes the implementation. Clause 6 documents the data model and interfaces. Clause 7 documents verification in the shape recommended by IEEE Std 829 and reports measured results. Clause 8 discusses results, trade-offs, limitations and risks. Clause 9 concludes and sets out future work. Appendices A–E hold the traceability matrix, API reference, configuration reference, execution log and reproduction procedure.

2 Overall Description

2.1 Product Perspective

MediTrack is a standalone, three-tier e-commerce system for a single pharmacy. It replaces a manual counter workflow in which stock levels and sales records are kept separately. The SRS fixes the architecture: a React client, a Node.js/Express API, a MongoDB database and a third-party sandbox payment gateway. The prototype is delivered as an academic system and is not integrated with any existing hospital, insurance or regulatory system.

Two architectural properties shape the rest of the design. First, the browser is never trusted with money or stock: prices, availability and totals are recomputed on the server on every read and write. Second, privileged operations are separated from customer operations at the API boundary rather than in the user interface, so hiding a button is never mistaken for a security control.

2.2 Product Functions

Table 2. Product functions and implementation status

Function group	Behaviour	Status
Catalogue	Public listing of medicines with name, manufacturing/expiry dates, price, stock and image; expired and zero-stock items are marked unavailable.	Implemented
Inventory administration	Staff/admin create, update and delete medicines, upload an image, mark prescription-only (Rx) items and set a refill cycle.	Implemented

Table 2 (continued)

Function group	Behaviour	Status
Replenishment	Medicines at or below a configurable watermark (default 10) are restored to a target level (default 100) by a background sweep or an on-demand action; expired stock is excluded.	Implemented
Cart	Guest cart keyed by a signed httpOnly cookie (only its SHA-256 hash stored) and account cart; guest cart merged on sign-in; quantities bounded by live stock.	Implemented
Pricing	Server-side subtotal, 5% tax, delivery fee and free-delivery threshold, all computed in integer paise and snapshotted on the order.	Implemented
Checkout and orders	Authenticated checkout with a validated shipping address; atomic per-line stock reservation with rollback; seven-state order lifecycle; cancellation and expiry release stock exactly once.	Implemented
Payments	Sandbox card (Luhn and expiry checks), UPI, net banking (demo banks) and cash on delivery; declines are retryable; only brand/bank and last four digits are retained.	Implemented
MediPoints	Append-only ledger; 1 point per ₹100 of medicine subtotal, credited once per order after a confirmed payment; balance and history exposed to the customer.	Implemented
Accounts and sessions	Customer self-registration; JWT access tokens (15 min) plus rotating refresh sessions (30 days) with reuse detection; password change revokes all sessions.	Implemented
Authorisation	Customer/staff/admin roles mapped to permissions; every protected route checks a permission, not a UI flag.	Implemented
Back office	Dashboard counters, order administration and fulfilment transitions, customer statistics, invite-based provisioning, audit-log viewer.	Implemented
Audit	Append-only trail for stock, order, role, invite, sign-in, password and replenishment events; no update/delete API exists.	Implemented
Cart soft holds	Reserve stock for a short window when an item is added to the cart, releasing automatically on expiry.	Partial — reservation begins at checkout
Prescriptions	Upload a prescription, staff verification queue, checkout blocked until verified, encrypted storage.	Planned (Phase D)
Wishlist	Independent saved list with move-to-cart.	Planned (Phase B)
Refill reminders	Reminder derived from purchase history that routes to a teleconsultation request rather than automatic re-dispatch.	Planned (Phase E)
QR/barcode	Simulated scan interaction (no hardware).	Planned (Phase F)

2.3 User Classes and Characteristics

The SRS names three user classes: Customer, Inventory Manager and Auditor. The implementation realises the first two as the customer and staff roles and covers the Auditor through the administrator's read-only audit view, because the Auditor is an external reviewer rather than a runtime actor of the store. Table 3 records the mapping, which must stay visible in any compliance discussion.

Table 3. User classes and their mapping to implemented roles

SRS user class	Implemented role	Capabilities
Customer	`customer`	Browse catalogue; manage cart; sign in; upload prescriptions (planned); place and pay for orders; view own orders and MediPoints.
Inventory Manager	`staff`	All stock operations, image upload, order fulfilment transitions, customer list, prescription verification (planned), manual replenishment.
Auditor	`admin` (audit view)	Read the audit log, roles and invitations. Admin additionally manages users, invites and role changes.
Anonymous visitor	no account	Browse catalogue and keep a cookie-identified cart; checkout requires sign-in.

2.4 Operating Environment

The runtime environment fixed by SRS Clause 4.7 and confirmed on the development machine is Node.js 18 or later (measured: Node.js 22.22.3, npm 10.9.8), MongoDB 6 or later (local instance or Atlas cluster), a modern browser (Chrome, Firefox, Edge or Safari) and HTTPS for all client-server traffic in production. The frontend is built with Vite 5 and can be served from any static host; the SRS names Netlify for the prototype. Uploaded medicine images are served from the API host under `/uploads`.

2.5 Design and Implementation Constraints

Table 4. Constraints and how they are respected

ID	Constraint (SRS)	Implementation response
CON-1	One academic semester, three-member team	Tracer-bullet delivery in phases; about 13.5k lines of code including 2.5k lines of tests, so each change stays reviewable.
CON-2	Sandbox payments only, no real transactions	Deterministic mock provider (`PAYMENT_PROVIDER=mock`); the provider adapter accepts a real gateway later without route changes.
CON-3	QR/RFID simulated, no hardware	No hardware dependency exists in the code path; a software simulation is planned (Phase F).
—	Single-instance prototype deployment	Rate limiting is in-memory; the README records the need for a shared store when scaling out.
—	No replica-set guarantee (standalone MongoDB)	Consistency relies on single-document conditional updates rather than multi-document transactions.

2.6 Assumptions and Dependencies

- ASSUME-1: standing refill authorisation is not available, so reminders route to assisted teleconsultation (FR-2.5).
- ASSUME-2: a stable internet connection exists during checkout.
- ASSUME-3: doctors are available for teleconsultation when a refill is due.
- The prototype assumes a single pharmacy with one stock pool; batch-level inventory is not modelled (see Clause 8.4).
- Dependencies: Node.js and npm; MongoDB (local or Atlas); a browser. No third-party service is required, because the payment provider is built in.

3 Specific Requirements

Requirements are taken in substance from SRS v1.0 and keep its identifiers and priority labels (High = required for the core demonstration, Medium = strengthens the build, Low = stretch). Each row carries the verified implementation status as of this revision, so the clause doubles as a compliance statement. Appendix A expands the same information into a full traceability matrix.

3.1 External Interface Requirements

Table 5. External interfaces

Interface	Specification (SRS)	Implementation
User interface	React customer interface; Inventory Manager console; cart and wishlist; prescription upload and status; checkout and payment status; purchase history and MediPoints.	React 18 + React Router 6 SPA with 16 page views (9 storefront, 7 back office) and 9 shared components; prescription upload and wishlist pending.
Hardware interface	QR/barcode simulated; no RFID or scanner hardware required.	No hardware dependency; simulation planned (Phase F).
Software interface	MongoDB 6+ for persistence; sandbox payment provider such as Razorpay or Stripe.	Mongoose 7 over MongoDB; a provider adapter (`lib/paymentGateway.js`) with a built-in mock provider implementing the same contract a real gateway would.
Communication interface	HTTPS for all client-server communication; JWT bearer tokens for authenticated requests.	Helmet security headers, HTTPS redirect and trust proxy in production; `Authorization: Bearer <token>`; Secure, httpOnly, SameSite=Lax cookies.
Data interface	Users, medicines/inventory, cart/wishlist, prescriptions, orders, payments, audit logs, MediPoints ledger.	Eight collections implemented; prescriptions and a separate wishlist collection pending (wishlist is Phase B, prescriptions Phase D).

3.2 Functional Requirements

Table 6. Functional requirements, priority and verified status

ID	Prio.	Requirement (SRS)	Status	Implementation / test evidence
FR-1.1	High	Replenish inventory when stock reaches the watermark of 10 units, restoring it to 100 units.	Implemented	`lib/inventory.js`; startup and 60 s sweeps in `server.js`; `POST /api/admin/inventory/replenish`; `tests/inventory.test.js`.
FR-1.2	High	Create a short-lived soft hold when an item is added to the cart; stock check and reservation performed atomically.	Partial	Atomic check-and-reserve exists but starts at checkout, not at add-to-cart: `lib/orderService.js` (conditional `findOneAndUpdate`); `tests/cart-orders.test.js`. Cart-level holds are Phase C.
FR-1.3	High	Automatically release an expired soft hold and make the inventory available again.	Partial	Expired order reservations are released (`expireStaleOrders`) exactly once via an atomic status transition. No cart-hold sweeper yet.
FR-1.4	High	Identify prescription-only medicines in the cart and prevent billing until the prescription is verified.	Planned	Data foundation only: `Medicine.prescriptionRequired` with a staff-editable flag and an Rx badge in the console. Checkout gate is Phase D.
FR-1.5	Medium	Maintain a wishlist independently of the cart; allow moving items from wishlist to cart.	Planned	Phase B: Wishlist collection, `/api/wishlist` routes, storefront page and navigation badge.
FR-1.6	High	Allow new customers to create an account using name, e-mail address and password.	Implemented	`routes/auth.js` — public registration is customer-only, privileged roles require an invite; `tests/api.test.js`.
FR-1.7	High	Allow registered customers to log in with their registered e-mail and password.	Implemented	`routes/auth.js`, `lib/sessions.js`; `tests/api.test.js`, `tests/sessions.test.js`.
FR-2.1	High	Block checkout for prescription-only medicines until the uploaded prescription has been verified.	Planned	Phase D: verification queue, checkout gate and `order.prescription` reference.
FR-2.2	High	If payment fails or is interrupted, leave inventory and order state unchanged; no partial decrement.	Implemented	Stock is reserved once at order creation; a declined payment only updates payment fields. A failed multi-line reservation rolls back every earlier line. `tests/cart-orders.test.js`.
FR-2.3	High	Support UPI, debit card, credit card and net banking through the sandbox gateway.	Implemented	`lib/providers/mockPayment.js`: card (debit/credit, Luhn + expiry + CVV), UPI, net banking (five demo banks plus a declining bank), cash on delivery. `tests/cart-orders.test.js`.

Table 6 (continued)

ID	Prio.	Requirement (SRS)	Status	Implementation / test evidence
FR-2.4	Medium	Credit MediPoints only after successful confirmation of payment.	Implemented	`lib/medipoints.js`, `models/MediPointsEntry.js`, `GET /api/me/medipoints`; one ledger entry per order enforced by a unique index; `tests/medipoints.test.js`.
FR-2.5	Medium	Route refill reminders to assisted teleconsultation booking rather than automatic re-dispatch.	Planned	Data foundation only: `Medicine.refillDays` (1–365, default 30). Reminder and teleconsultation request are Phase E.
FR-2.6	High	Maintain an immutable purchase history supporting reminders and repeat purchase.	Implemented	Orders are snapshots (name, image, unit price, line total, totals, address) and no route updates or deletes them: `models/Order.js`. A schema-level immutability guard is scheduled for Phase F.

3.3 Non-Functional Requirements

Table 7. Non-functional requirements and verified status

ID	Prio.	Requirement (SRS)	Status	Implementation / evidence
NFR-1.1	High	The server shall enforce role-based access control for every protected request.	Implemented	Route-level `requireAuth` / `requireRole` / `requirePermission`; roles map to permissions in `lib/permissions.js`; the SPA receives the same list so it never hard-codes role logic. 401/403 paths are asserted in `tests/api.test.js`, `tests/admin.test.js` and `tests/inventory.test.js`.
NFR-1.2	High	Use bcrypt for password hashing and JWT-based authentication with session expiry.	Implemented	bcryptjs with ten rounds and `select:false` on the hash; 15-minute access tokens; 30-day rotating refresh sessions stored only as SHA-256 hashes; `tokenVersion` invalidates outstanding tokens on password change or “sign out everywhere”; `tests/sessions.test.js`.
NFR-1.3	High	The audit log shall be append-only; no update or delete operations for audit records.	Implemented	`lib/audit.js` exposes `recordAudit` and a paginated read; no route updates or deletes `auditlogs`. A database-level write guard is scheduled for Phase F.

Table 7 (continued)

ID	Prio.	Requirement (SRS)	Status	Implementation / evidence
NFR-1.4	High	Payment tokens and prescription data shall be encrypted or tokenised at rest.	Partial	No card number or CVV is persisted: only brand, last four digits and a generated transaction id. Prescription encryption is Phase D (AES-256-GCM with a key from the environment, stored outside the static upload directory).
NFR-2.1	High	Inventory reservation shall be atomic to prevent lost updates and overselling.	Implemented	Per-line conditional update `{stock: {$gte: qty}}` → `$inc {stock: -qty}` with rollback of earlier lines; atomic status transitions make every stock restore single-shot. `tests/cart-orders.test.js` covers over-selling, rollback, cancel and expiry.
NFR-2.2	Medium	The checkout interface shall display the remaining stock-hold time and prescription verification status.	Partial	The payment page shows the minutes left on the reservation (`minutesLeft(order.expiresAt)`). A cart-level countdown and Rx verification status arrive with Phases C and D.
NFR-2.3	Low	Designed with a 24×7 operational goal; production uptime guarantees out of scope.	Goal	Stateless API, idempotent payment and replenishment operations and graceful shutdown support the goal; no monitoring, backup or SLA evidence exists for an academic prototype.

3.4 Requirements Interpretation and Traceability

Summary. Of the thirteen functional requirements, seven are implemented and verified by tests, two are partially implemented and four are planned. Of the seven non-functional requirements, four are implemented, two are partial and one is a stated goal. No requirement has been dropped or silently re-scoped.

Table 8. Areas where the SRS needs an explicit reading

Requirement	Ambiguity or mismatch	Reading adopted in this build
FR-1.1	The SRS does not say when replenishment runs, only that it must happen at the watermark.	Implemented as a supplier-delivery simulation: a sweep (startup plus interval) and an explicit Inventory Manager action. It is deliberately not inline in checkout, so a sale never changes stock it did not reserve. Watermark and target are configuration, not constants.
FR-1.2 / FR-1.3	The SRS says “short-lived soft hold” without a duration, and the earlier report draft assumed a ten-minute figure that the SRS v1.0 does not state.	The current mechanism is an order-level reservation of `ORDER_RESERVATION_MINUTES` (default 30). Phase C adds a cart-level hold (`CART_HOLD_MINUTES`, default 15) on a `Medicine.held` counter with a sweeper, which is the literal reading of FR-1.2/1.3.
FR-2.3	“Debit card” and “credit card” are listed separately.	One card method serves both; the provider validates the number, expiry and CVV identically and stores only the brand and last four digits.

Table 8 (continued)

Requirement	Ambiguity or mismatch	Reading adopted in this build
FR-2.6	"Immutable" is not defined at storage level.	Immutability is achieved by snapshotting commercial data into the order and by exposing no mutating route. A write-guard in the model is planned as defence in depth.
User classes	The SRS names an Auditor; the code base has customer/staff/admin.	The Auditor is served by the administrator's read-only audit view. A dedicated auditor role with `audit:read` only is a low-cost future addition.
NFR-2.3	A 24×7 goal cannot be demonstrated by a single-host prototype.	Recorded as a design goal with the operational work listed in Clause 9.2; no uptime claim is made.

4 System Design Description

This clause follows the design-view structure of IEEE Std 1016-2009. Each view addresses one or more stakeholder concerns; together the views constitute the design description. Implementation detail is kept for Clause 5 so that the design remains readable on its own.

4.1 Design Stakeholders and Concerns

Table 9. Stakeholders, concerns and the views that address them

Stakeholder	Concern	Addressed in
Course evaluator / guide	Requirements coverage and evidence of engineering discipline	Clauses 3, 7, 8; Appendices A, D
Customer	Accurate availability, predictable pricing, safe checkout, visible order state	4.2.3, 4.2.7, 4.2.8, 6.3
Inventory Manager (staff)	Correct stock, low-stock visibility, order fulfilment, traceable actions	4.2.2, 4.2.9, 4.2.8
Administrator	Privilege separation, provisioning of privileged accounts, auditability	4.2.4, 4.2.6, 5.5
Successor development team	Modularity, test coverage, documented conventions	4.2.2, 5, 7, Appendix E
Auditor (predecessor group)	Evidence that privileged actions leave a non-repudiable record	4.2.6, 5.5, 6.1

4.2 Design Views

4.2.1 Context view

MediTrack has four external actors and two external systems. The actors are the anonymous visitor, the customer, the Inventory Manager and the administrator. The external systems are the MongoDB deployment and the payment gateway, which is reachable only through the provider adapter.

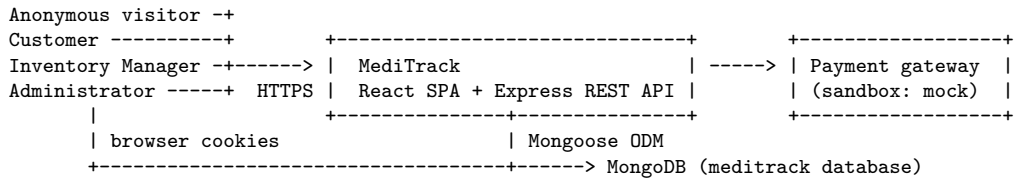


Fig. 1. Context view

Figure 2 complements the context view with the information flow of the portal. The dashed line is the system boundary; thirteen processes inside it exchange data with the central MediTrack database, and each labelled box is information entering or leaving a process. The payment gateway is the only third-party system and is drawn with a double boundary. The processes for the wishlist, prescription verification, cart soft holds and refill reminders show the target design of SRS v1.0; their current status is given in Table 2 and Clause 8.2.

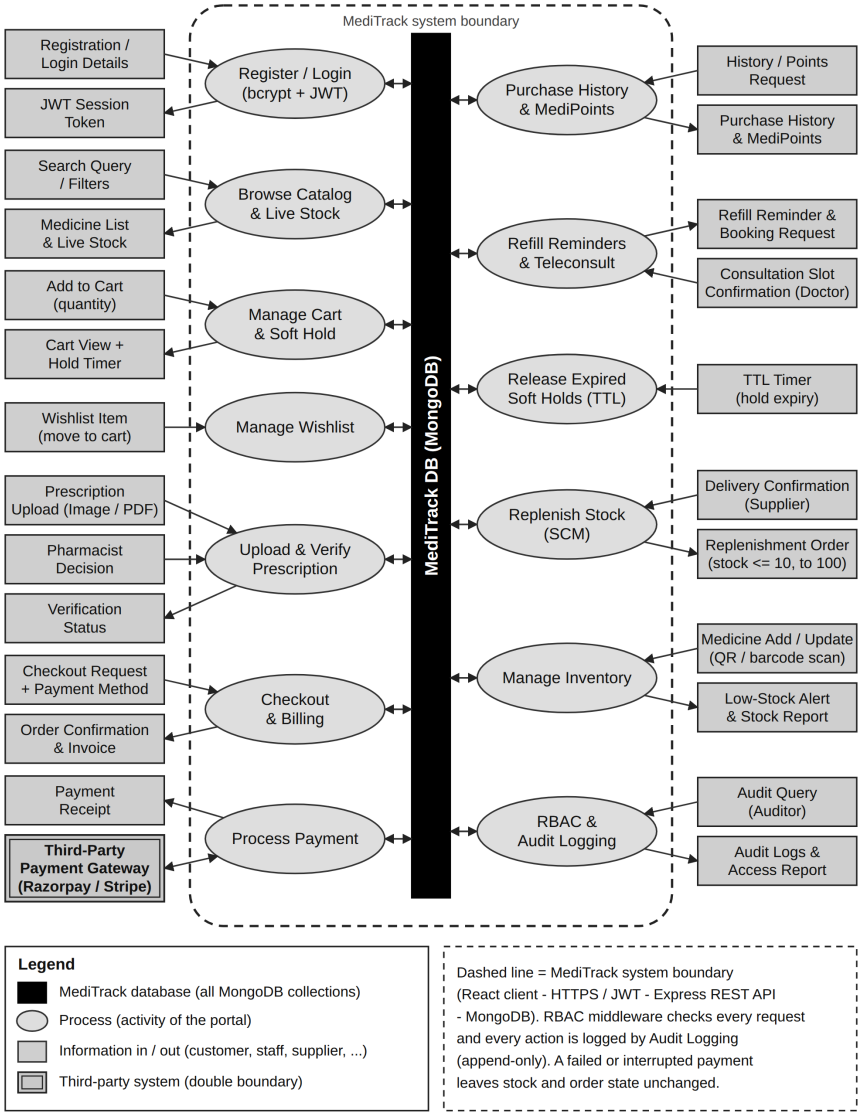


Fig. 2. Information flow diagram of the MediTrack portal

4.2.2 Composition view

The API is decomposed into routers, shared libraries, models and middleware. The rule applied throughout is that a route validates input and authorises the caller, a library implements business rules, and a model declares data shape and indexes. Nothing in a route computes money or stock directly.

Table 10. Composition view: components and responsibilities

Component	Responsibility	Key files
API entry point	Security headers, CORS, cookies, route mounting, central error normalisation, startup and shutdown, background sweep	`server.js`
Routers (7)	Authentication, back-office auth, back-office management, medicines, cart, orders, customer account	`routes/`
Catalogue domain	Medicine schema, availability rules, replenishment watermark logic	`models/Medicine.js`, `lib/availability.js`, `lib/inventory.js`
Cart domain	Server-side cart documents, signed cookie identity, guest merge, live serialisation	`models/Cart.js`, `lib/cartSession.js`
Order domain	Cart validation, atomic reservation and rollback, expiry, order snapshots, totals	`models/Order.js`, `lib/orderService.js`, `lib/pricing.js`
Payment domain	Provider registry and charge contract; deterministic sandbox rules	`lib/paymentGateway.js`, `lib/providers/mockPayment.js`
Loyalty domain	Earning rule, idempotent credit, balance aggregation	`models/MediPointsEntry.js`, `lib/medipoints.js`
Identity domain	Accounts, bcrypt hashing, access tokens, refresh sessions, invitations	`models/User.js`, `models/Session.js`, `models/AdminInvite.js`, `lib/sessions.js`, `lib/invites.js`, `middleware/auth.js`
Governance	Audit recording and reading, role-to-permission map, rate limits, request guards	`lib/audit.js`, `lib/permissions.js`, `middleware/rateLimit.js`, `middleware/requestGuards.js`
Storefront SPA	Routing, auth and cart contexts, pages and shared components, fetch wrapper with silent token refresh	`frontend/src/`

4.2.3 Logical view

Figure 3 shows the layering. The frontend holds no business truth: it renders what the API returns and re-fetches after every mutation. The API layer is the only place where prices, availability and permissions are decided. Persistence is reached exclusively through Mongoose models, which also carry the indexes the concurrency rules depend on.

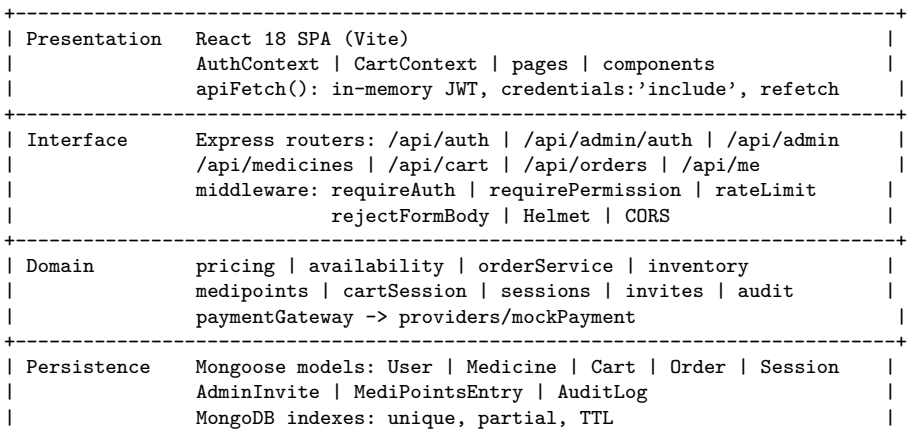


Fig. 3. Logical layered view

Table 11. Layer responsibilities and prohibitions

Layer	May do	Must not do
Presentation	Render, validate for usability, hold the access token in memory, call the API	Decide prices, stock, permissions or order state; persist tokens in `localStorage`
Interface	Parse and validate input, authorise, translate errors to status codes	Contain business rules that are also needed by another route
Domain	Own invariants, atomicity, money arithmetic, provider selection	Depend on Express request/response objects (only an optional audit context is passed)
Persistence	Declare schema, validation, indexes, serialisation to id	Hold secrets in plaintext (password and token hashes only, `select:false`)

4.2.4 Dependency view

Dependencies point downwards only. Routers may use domain libraries and models; domain libraries may use models and other domain libraries; models depend on nothing but Mongoose, plus the permission map used to serialise a user. The payment adapter is the single seam to an external provider, and it is inverted: the route knows only `charge()`.

Table 12. Dependency view (selected edges)

From	To	Purpose
`routes/orders.js`	`lib/orderService.js`, `lib/paymentGateway.js`, `lib/medipoints.js`	Create orders, charge, credit points
`lib/orderService.js`	`models/Order.js`, `models/Medicine.js`, `lib/pricing.js`, `lib/paymentGateway.js`	Snapshot, reserve, price, record provider
`lib/inventory.js`	`models/Medicine.js`, `lib/audit.js`, `lib/availability.js`	Watermark sweep and audit
`lib/medipoints.js`	`models/MediPointsEntry.js`, `lib/pricing.js`	Earning rule and idempotent credit
`middleware/auth.js`	`models/User.js`, `lib/permissions.js`	Token verification and permission checks
`server.js`	all routers, `lib/cartSession.js`, `lib/migrations.js`, `lib/inventory.js`	Composition root
`frontend/src/lib/api.js`	REST API	Single fetch wrapper, token refresh, grouped API objects

4.2.5 Information view

The persistent information view is specified in Clause 6.1 (collections, fields and indexes) and Clause 6.2 (relationships). The design rule is that each collection owns exactly one entity, references are stored as ObjectIds with declared indexes, and commercial values that must not change (unit price, totals, address) are copied into the order at checkout rather than read through the reference.

Figure 4 presents the domain class model of the SRS with the principal attributes and operations of each class. User is specialised into Customer, Inventory Manager and Auditor; a Customer owns carts, a wishlist, orders, a MediPoints ledger and prescriptions, and an Order can have several payment attempts. The model is the design target. In the current build the Auditor is served by the administrator's audit view (Table 3), Payment is embedded in the order, and the Wishlist, Prescription, RefillReminder and ConsultationBooking classes are planned (Table 2, Clause 8.2).

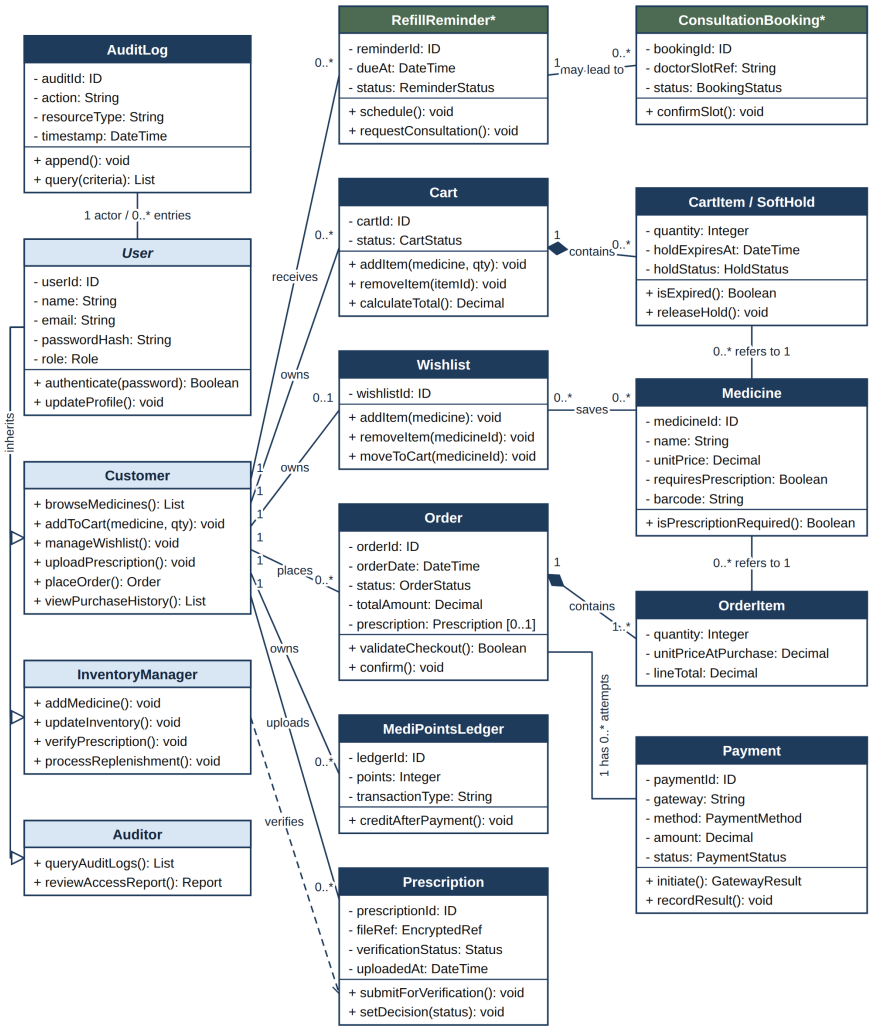


Fig. 4. Domain class model (UML) of MediTrack

4.2.6 Interface view

Table 13. Interface view: access rules per interface

Interface	Authentication	Authorisation rule
`POST /api/auth/register`, `/api/auth/login`	None	Public; role forced to `customer`; rate limited; JSON only
`GET /api/medicines`, `/api/medicines/:id`	None	Public read; response exposes only catalogue fields
`POST|PATCH|DELETE /api/medicines...`	Bearer JWT	`catalog:write` (staff, admin)
`/api/cart...`	Optional bearer JWT	Guest cart via signed cookie; account cart when signed in; merge requires a token
`/api/orders...`	Bearer JWT	Owner only; foreign order ids return 404 rather than 403 to avoid leaking existence
`/api/me/medipoints`	Bearer JWT	Self only
`/api/admin...`	Bearer JWT	`orders:read:all` for operational views; `users:manage`, `invites:manage`, `audit:read` for administration
`/api/admin/auth/*`	None	Public back-office login (customers rejected with 403) and invite acceptance with a single-use token
`/uploads/*`	None	Static medicine images only; filenames are generated, never taken from the client

4.2.7 Interaction view

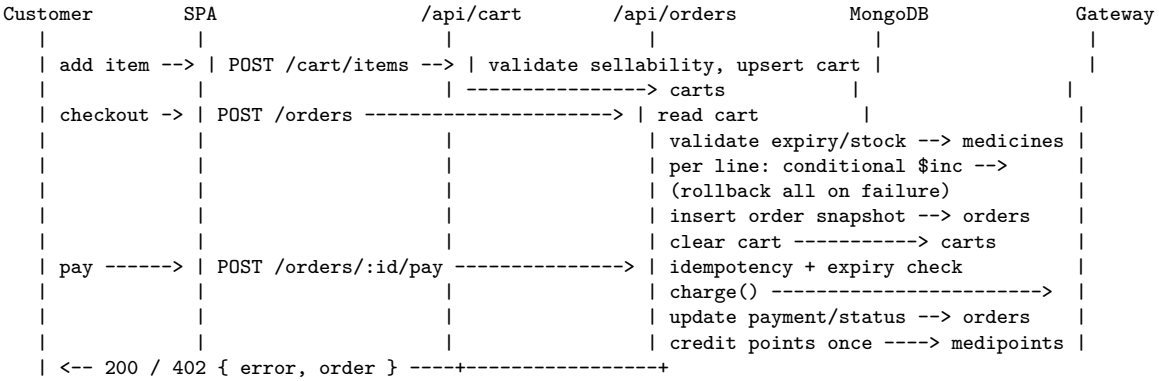


Fig. 5. Checkout and payment interaction

Table 14. Interaction view: principal use cases

Use case	Entry point	Behaviour and failure handling
Browse catalogue	`GET /api/medicines`	Newest first; expired and zero-stock rows are returned but flagged unavailable by the cart layer.
Manage cart	`/api/cart...`	Guest or account cart; quantity bounded by live stock and a per-line maximum; 409 on expiry or insufficient stock; 415 for form-encoded bodies.
Place order	`POST /api/orders`	Validates the whole cart before touching stock; reserves line by line; rolls back on the first failure; snapshots prices; clears the cart; sets an expiry deadline.
Pay order	`POST /api/orders/:id/pay`	Rejects unsupported methods (400), expired orders (409) and non-payable states (409); returns 402 with the order on decline; 200 without re-charging when already paid.
Cancel / expire	`POST /api/orders/:id/cancel`	Atomic status transition claims the order, then restores stock; a second call is a no-op returning the cancelled order.

Table 14 (continued)

Use case	Entry point	Behaviour and failure handling
Fulfil order	`PATCH /api/admin/orders/:id/status`	Only forward transitions from a fixed map (paid→processing→shipped→delivered; pending_payment→cancelled); each change is audited.
Restock	`POST /api/admin/inventory/replenish`	Refills every unexpired medicine at or below the watermark, reports each change and audits it.
Provision staff	`POST /api/admin/invites`	Single-use token returned once, only its SHA-256 hash stored, expires after 72 hours by default; acceptance creates the account with the invited role.

4.2.8 State dynamics view

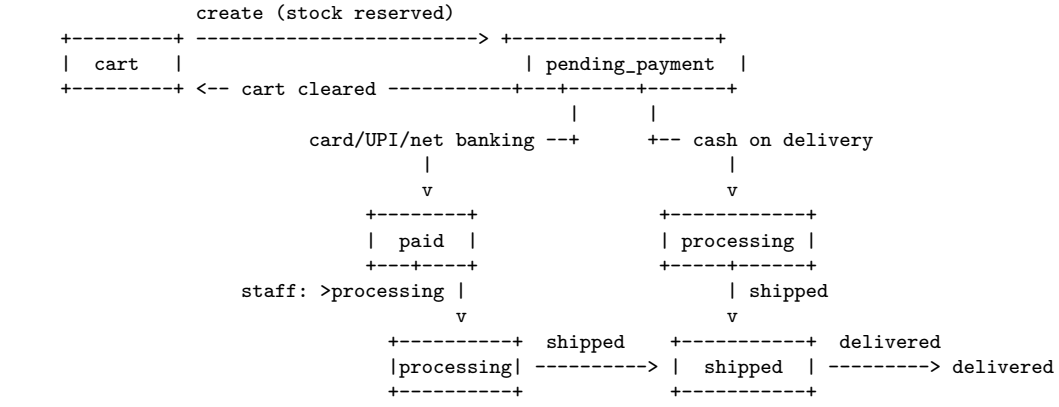


Fig. 6. Order and stock state machine

Table 15. Order states, allowed transitions and effects

State	Meaning	Allowed next states	Stock / points effect
`pending_payment`	Created, stock reserved, awaiting payment	`paid`, `processing` (COD), `cancelled`, `expired`	Stock held until the deadline; no points
`paid`	Card / UPI / net banking captured	`processing`	Stock stays decremented; points credited once
`processing`	Confirmed, cash to be collected on delivery	`shipped`	Payment still pending; no points until collected
`shipped`	Dispatched	`delivered`	No stock change
`delivered`	Fulfilled	—	No stock change
`cancelled`	Cancelled before payment	—	Stock restored exactly once
`expired`	Reservation lapsed	—	Stock restored exactly once

4.2.9 Algorithm view

Three algorithms carry the correctness of the system. They are written as single-document conditional updates, because the prototype must also run on a standalone MongoDB instance without multi-document transactions.

A — atomic reservation with rollback (FR-1.2, NFR-2.1). The condition and the decrement travel in one database operation, so two concurrent checkouts can never both pass a

stock check.

```
for each cart line:
    updated = medicine.findOneAndUpdate(
        { _id: id, stock: { $gte: quantity } },      // condition re-evaluated atomically
        { $inc: { stock: -quantity } },
        { new: true })
    if not updated:                                   // someone sold the last unit first
        restore every previously reserved line        // rollback
        throw 409 "...no longer has N unit(s) in stock"

totals  = computeTotals(lines)                        // integer paise, server-side only
order   = insert snapshot + expiry deadline
cart.items = []
```

B — watermark replenishment (FR-1.1). The sweep is idempotent: after a row is refilled it no longer satisfies the condition, so a second concurrent sweep matches nothing.

```
candidates = medicines.where(stock <= LOW_STOCK_WATERMARK, exp >= today)
for each candidate:
    updated = medicine.findOneAndUpdate(
        { _id: candidate._id, stock: { $lte: LOW_STOCK_WATERMARK } },
        { $set: { stock: REPLENISH_TO } },
        { new: true })
    if not updated: continue                          // another sweep won the race
    audit("medicine.replenish", before={stock}, after={stock})
```

C — MediPoints credit (FR-2.4). Eligibility is decided from the payment state, and the database enforces "once per order".

```
if order.payment.status != 'succeeded': return null    // declined, cancelled or COD unpaid
points = floor(subtotal_in_paise / 10000) * MEDIPOINTS_PER_100
try: insert ledgerEntry { user, order, type:'earn', points }
catch duplicate-key on order: return null              // already credited -- harmless replay
```

4.2.10 Resource view

- **Process model.** One Node.js process serving HTTP and one unref'd interval timer for replenishment; no worker queue, no external scheduler.
- **Memory.** Bounded by request size (JSON body default limits, 5 MB upload limit) and the in-memory rate-limit store, which grows with the number of distinct IP/e-mail pairs seen inside the window.
- **Storage.** MongoDB holds documents and indexes; uploaded images are files under `backend/uploads` and are deleted when the medicine is removed or its image replaced.
- **Timers and TTL.** Guest carts carry a TTL index (30 days); orders carry an application-level deadline because stock must be restored, not merely deleted.
- **Scaling limits.** Rate limiting and scheduled sweeps assume a single instance. Multiple instances would need a shared limiter store and an elected sweeper.

4.3 Design Rationale

Table 16 records the decisions that materially shape the system, the alternatives that were rejected, and the consequence the successor team inherits. It is written as a lightweight decision record so a future reviewer can see why, not only what.

Table 16. Design decisions and rejected alternatives

Decision	Alternative rejected	Rationale and consequence
Cart lives on the server; the cookie carries only a random token whose SHA-256 hash is stored	Cart in `localStorage` or in a readable cookie	XSS cannot read the cart, the cookie cannot be forged (HMAC), and prices never come from the browser. Consequence: one database round trip per cart read, acceptable at this scale.
All money arithmetic in integer paise	Floating-point rupees	Eliminates rounding drift between subtotal, tax and total. Consequence: conversions at the boundary; rates and rounding are recorded on the order so old bills stay explainable.
Single-document conditional updates for stock and status	Multi-document transactions	Works on a standalone MongoDB (the common local setup) and still prevents lost updates. Consequence: multi-line reservations are compensated by application-level rollback instead of a transaction.
Commercial data copied into the order (snapshot)	Reading prices through the medicine reference at display time	FR-2.6: a later price change or deletion cannot rewrite history. Consequence: the order document is larger and intentionally denormalised.
Payment provider adapter with a built-in simulator	Calling one vendor SDK directly from the route	CON-2 keeps payments in sandbox; adding Razorpay or Stripe later means one new provider file and one environment variable. Consequence: the adapter contract must stay small (charge plus an optional redirect result).
Permissions, not role strings, enforced in routes	Scattered `role === 'admin'` checks	One map decides what a role may do, and the same list is sent to the SPA so the UI never invents its own rules. Consequence: adding a capability means editing one file.
Privileged accounts only by invitation; public registration is customer-only	An administrator sign-up code in configuration	Removes a shared secret from configuration and makes provisioning traceable. Consequence: the first administrator must be seeded by script.
Short access token in memory + rotating refresh cookie	A long-lived JWT in `localStorage`	NFR-1.2: a stolen access token expires quickly, and reuse of a rotated refresh token revokes the account's sessions. Consequence: more moving parts, covered by `tests/sessions.test.js`.
MediPoints as an append-only ledger with a unique index on order	A mutable points field on the user	Auditable earning history and database-enforced idempotency under retries. Consequence: the balance is an aggregation rather than a stored number.
Replenishment as a scheduled sweep plus an explicit action	Refilling inside the checkout code path	Keeps a sale honest: it returns exactly the stock it reserved. Consequence: stock can sit at the watermark until the next sweep, which is the intended supplier-delivery semantics.
Expired medicines excluded from replenishment	Refilling every low row	Refilling unsellable stock hides the problem from the Inventory Manager and inflates inventory value. Consequence: expired rows stay visible and flagged until removed.
Audit writes never throw	Failing the business action when logging fails	A logging fault must not turn a committed sale into an error. Consequence: audit gaps are possible in principle and are reported to the console for the operator.

5 Implementation

5.1 Technology Stack

Table 17. Technology stack (versions from the project manifests)

Layer	Technology	Role / notes
Client	React 18.3, React Router 6.30, Vite 5.4	Single-page application, route guards, build tooling
Server	Node.js 22.22.3 (runtime), Express 4.21	REST API, middleware chain, static uploads
Database	MongoDB 6+, Mongoose 7.8	Documents, schema validation, unique/partial/TTL indexes
Authentication	jsonwebtoken 9, bcryptjs 3	Access tokens, password hashing (10 rounds)
Security middleware	Helmet 8, express-rate-limit 8, cookie-parser 1.4, cors 2.8	Headers, brute-force limits, signed cookies, credentialed CORS
Uploads	Multer 2	Medicine images (JPEG/PNG/GIF/WEBP, 5 MB limit)
Testing	`node:test`, `node:assert/strict`, mongodb-memory-server 11	Real HTTP against a real Express app and an in-memory MongoDB
Configuration	dotenv 17	Environment-driven settings; production refuses to start without secrets

5.2 Repository Structure and Size

Table 18. Repository structure and measured size

Path	Lines	Contents
`backend/models/`	685	8 schemas: User, Medicine, Cart, Order, Session, AdminInvite, MediPointsEntry, AuditLog
`backend/routes/`	1,539	7 routers, 40 route handlers
`backend/lib/`	1,590	14 domain libraries plus the mock payment provider
`backend/middleware/`	238	Authentication/authorisation, rate limits, request guards
`backend/tests/`	2,551	8 suites and one shared helper (97 test cases)
`backend/scripts/`	302	`db:check`, `local-mongo`, `seed-admin`, `migrate-roles`, `replenish-inventory`
`frontend/src/pages/`	2,830	16 page views (9 storefront, 7 back office)
`frontend/src/components/`	751	9 shared components (navigation, dialogs, stepper, summaries, guards)
`frontend/src/context/`	336	AuthContext and CartContext
`frontend/src/lib/`	317	`apiFetch` wrapper, grouped API clients, formatting helpers
`frontend/src/App.css`	1,884	Single stylesheet, no CSS framework

5.3 Backend Modules

The API is composed in `server.js`, which applies Helmet, credentialed CORS, JSON parsing and a signed-cookie parser, serves `/uploads`, mounts the seven routers, terminates unknown `/api` paths with 404 and normalises every error (Mongoose validation, cast and duplicate-key errors, and Multer failures) into `{error}` with an appropriate status code. Multer's temporary files are removed when a request fails, so a rejected upload leaves no orphan on disk.

Catalogue and availability. `models/Medicine.js` declares name, manufacturing and expiry dates (stored as `YYYY-MM-DD` strings), price, integer stock, an optional image reference, creator, `prescriptionRequired` and `refillDays`. A `pre('validate')` hook rejects an expiry earlier than the manufacturing date, so an invalid period cannot be stored even if the client bypasses the form. `lib/availability.js` treats a medicine as sellable until the end of its expiry day and returns a human-readable reason when it is not.

Replenishment. `lib/inventory.js` reads its rules from the environment, refuses an invalid configuration (a refill target at or below the watermark would refill the same row forever) and

exposes `replenishLowStock()` and `startReplenishSweeper()`. The sweep runs once at startup and then on an interval; the timer is unreferenced so it never keeps a process or a test run alive, and it is cleared when the server closes. Every refill is audited with the before and after quantity.

Cart. `lib/cartSession.js` resolves the caller's cart from the JWT or from the signed `mt_cart` cookie (path `/api`, httpOnly, SameSite=Lax, Secure in production, 30-day lifetime). Only `sha256(token)` is stored. `serializeCart()` re-reads every medicine, so prices, stock and availability are current; it marks per-line issues and computes totals. Merging a guest cart into an account cart clamps each line to live stock and to the per-line maximum instead of failing the sign-in.

Orders and payments. `lib/orderService.js` validates the whole cart before touching stock, reserves line by line, rolls back on failure, snapshots items and totals, clears the cart and sets the expiry deadline. Order numbers take the form `MT-YYYY-XXXXXX` under a unique index with a bounded retry on collision. `routes/orders.js` enforces ownership, idempotent payment, expiry checks and cancellation. `lib/paymentGateway.js` is a small registry; the mock provider validates card numbers with the Luhn checksum plus expiry and CVV, accepts UPI with a valid VPA, accepts five demo banks for net banking and declines cash on delivery above ₹5,000.

MediPoints. `lib/medipoints.js` computes points from the subtotal in whole ₹100 blocks and inserts one ledger entry per order. A unique partial index on order turns a duplicate credit into a caught duplicate-key error, so a retried payment cannot earn twice.

Identity and sessions. `models/User.js` hashes with bcrypt (ten rounds) and hides the hash with `select:false`. `lib/sessions.js` issues a 32-byte refresh token in a signed httpOnly cookie, stores only its SHA-256 hash with a device label and expiry, rotates on every refresh, and revokes every session when a rotated token is replayed outside a short grace window (which covers two tabs refreshing at once). `tokenVersion` is embedded in access tokens so a password change or "sign out everywhere" invalidates outstanding tokens immediately.

Governance. `lib/permissions.js` maps roles to permissions (`catalog:read`, `catalog:write`, `orders:read:all`, `orders:write`, `customers:read`, `users:manage`, `invites:manage`, `audit:read`). `lib/audit.js` writes a best-effort entry (actor, actor e-mail, action, target, before/after, IP, user agent) and never throws. `middleware/rateLimit.js` limits failed sign-ins per IP and e-mail and registrations per IP; `middleware/requestGuards.js` rejects form-encoded bodies on JSON-only endpoints, closing the cross-site form-post path.

5.4 Frontend Modules

The SPA is a single-page React application with two providers at the root: AuthProvider (signed-in user, silent refresh on mount, logout, session-expired event) and CartProvider (cart state, merge on sign-in, add-to-cart dialog state). Routing distinguishes the storefront from the `/admin` console by pathname so the two layouts never mix.

Area	Modules	Notes
Storefront pages	Home, Login, Cart, Checkout, Payment, OrderConfirmation, Orders, AddMedicine (legacy), AdminRegister	Checkout requires a session; the cart itself works for guests
Back-office pages	AdminLogin, AdminDashboard, AdminStock, AdminOrders, AdminCustomers, AdminInvites, AdminAudit	Guarded by role; invites and audit are admin-only
Shared components	Navbar, ProtectedRoute, MedicineList, MedicineForm, Modal, AddToCartDialog, QuantityStepper, OrderSummary, AdminLayout	Modal uses the native `<dialog>` element for focus trapping and ESC handling

Area	Modules	Notes
API client	`lib/api.js`	Holds the access token in a module variable, attaches it to every request, retries once after a silent refresh, and groups endpoints into `adminApi`, `cartApi`, `ordersApi`, `accountApi`
Formatting	`lib/format.js`	Currency (₹, en-IN), dates, minutes remaining, status labels

Two frontend behaviours are worth recording because they encode requirements. First, the payment page renders a countdown from `order.expiresAt` and re-fetches the order on a decline, so the customer sees the true state after a 402 (NFR-2.2, partially). Second, the stock page reads the watermark and refill target from the dashboard summary rather than hard-coding 10, so the badge and the rule cannot drift apart.

5.5 Security Implementation

Table 19. Security controls mapped to requirements

Control	Implementation	Requirement
Password storage	bcryptjs, ten rounds, hash excluded from queries by default and from JSON serialisation	NFR-1.2
Access tokens	15-minute JWT with subject, e-mail, role, `tokenVersion` and a unique `jti`; verified on every protected request with the user re-read from the database	NFR-1.2
Refresh sessions	Signed httpOnly cookie, SHA-256 hash at rest, rotation, reuse detection, per-device revocation, global revocation	NFR-1.2
Authorisation	`requireAuth` → `requirePermission` / `requireRole`; roles re-read per request, so a demotion takes effect on the next call	NFR-1.1
Privileged provisioning	Single-use, expiring invites; only the hash is stored; public registration cannot request a privileged role	NFR-1.1
Last-admin guard	Self-demotion rejected and the final administrator cannot be demoted	NFR-1.1
Brute-force protection	Failed sign-ins limited per IP and e-mail, registrations per IP, configurable window and thresholds	NFR-1.1
CSRF posture	JWT in an `Authorization` header (not a cookie) for state changes; SameSite=Lax cookies; JSON-only mutations; unknown origins rejected by CORS	NFR-1.1
Transport and headers	Helmet defaults (CSP disabled deliberately because the API returns JSON and images), HSTS, nosniff, no `X-Powered-By`; HTTPS redirect and trust proxy in production	SRS 4.6
Payment data minimisation	Card number and CVV are validated and discarded; only brand, last four digits and a generated transaction id are stored	NFR-1.4
Audit trail	Append-only writes for stock, replenishment, order status, roles, invites, sign-in and password changes; no update/delete endpoint	NFR-1.3
Secrets management	All keys from environment variables; production refuses to start without `JWT_SECRET` and `COOKIE_SECRET`; development fallbacks are loudly warned about	SRS 4.7
File upload	MIME allow-list, 5 MB limit, generated filenames, deletion on replace/remove, orphan cleanup on request failure	—

5.6 Configuration and Tooling

Behaviour that an operator may reasonably tune is environment-driven rather than hard-coded. Appendix C lists the full set; the ones that carry requirements are the pricing rates, the reservation window, the replenishment watermark and interval, the MediPoints rate and the authentication lifetimes.

Five maintenance scripts sit behind npm scripts: `db:check` verifies a connection string and prints the target database, `db:local` starts a local MongoDB without an installation, `seed:admin` creates or promotes the first administrator, `migrate:roles` applies pending one-time data migrations, and `inventory:replenish` runs a single replenishment pass for a cron job or a demonstration. Migrations are also applied automatically at startup and are recorded in a `migrations` collection so they never run twice.

6 Data Design and Interfaces

6.1 Data Model

The database is named `meditrack` by default. Eight application collections implement the SRS entities; the SRS "Payments" entity is embedded in the order because a payment has no independent identity or history in this prototype, and "Prescriptions" is not yet implemented. A ninth collection, `migrations`, is internal bookkeeping for one-time data fixes.

Table 20. Collections, key fields and purpose

Collection	Key fields	Purpose / SRS entity
`users`	name, email (unique, lowercase), passwordHash (`select:false`), role (customer | staff | admin), tokenVersion, passwordChangedAt, timestamps	Accounts and roles — SRS Users
`medicines`	name, mfd, exp (YYYY-MM-DD), price, stock (integer), prescriptionRequired, refillDays (1–365), image, createdBy → users, timestamps	Catalogue plus inventory — SRS Medicines/Inventory
`carts`	user → users (unique partial), tokenHash (unique partial, `select:false`), items[{ medicine → medicines, quantity }], expiresAt (TTL)	Guest and account carts — SRS Cart/Wishlist (wishlist pending)
`orders`	orderNumber (unique), user → users, items[{ medicine, name, image, unitPrice, quantity, lineTotal }], shippingAddress{…}, totals{subtotal, tax, deliveryFee, total, itemCount, currency, taxRate, freeDeliveryOver}, status, payment{provider, method, status, transactionId, last4, brand, failureReason, paidAt}, expiresAt, timestamps	Immutable purchase snapshots — SRS Orders + Payments
`sessions`	user → users, tokenHash (unique, `select:false`), userAgent, ip, lastUsedAt, expiresAt (TTL), revokedAt, revokedReason, replacedBy → sessions	Refresh sessions with rotation and reuse detection
`admininvites`	email, role (admin | staff), tokenHash (unique, `select:false`), createdBy → users, expiresAt, acceptedAt, acceptedBy, revokedAt	Single-use provisioning of privileged accounts
`medipointsentries`	user → users, order → orders, type (earn | spend | adjust), points (signed integer), reason, created_at	Append-only loyalty ledger — SRS MediPoints Ledger
`auditlogs`	actor → users, actorEmail, action, targetType, targetId, before, after, ip, userAgent, created_at	Append-only audit trail — SRS AuditLogs
`migrations`	_id (migration id), appliedAt, detail	Internal: guarantees a data fix runs once

Table 21. Indexes and the invariant each one enforces

Collection	Index	Purpose
`users`	`{email: 1}` unique	One account per address
`medicines`	`{created_at: -1}`	Catalogue ordering (newest first)
`carts`	`{user: 1}` unique partial; `{tokenHash: 1}` unique partial; `{expiresAt: 1}` TTL	One cart per account and per guest token; abandoned guest carts expire
`orders`	`{orderNumber: 1}` unique; `{user: 1, created_at: -1}`; `{status: 1, expiresAt: 1}`	Unique order number, fast history, fast expiry sweep
`sessions`	`{tokenHash: 1}` unique; `{user: 1, revokedAt: 1}`; `{expiresAt: 1}` TTL	No duplicate tokens; device listing; automatic cleanup
`admininvites`	`{tokenHash: 1}` unique; `{email: 1, created_at: -1}`; `{expiresAt: 1}`	Single-use tokens; pending-invite listing
`medipointsentries`	`{order: 1}` unique partial; `{user: 1, created_at: -1}`	FR-2.4: at most one credit per order; fast balance history
`auditlogs`	`{created_at: -1}`; `{action: 1, created_at: -1}`	Newest-first listing and filtering by action

6.2 Relationships

```
users 1 ---- 0..1 carts           (user reference, unique when present)
users 1 ---- 0..* orders          (order.user)
users 1 ---- 0..* sessions        (session.user)
users 1 ---- 0..* medipointsentries
users 1 ---- 0..* admininvites    (createdBy)  and 0..1 accepted invite (acceptedBy)
users 1 ---- 0..* medicines       (createdBy, for provenance)
medicines 1 -- 0..* carts.items   and  0..* orders.items  (reference + snapshot copy)
orders 1 ---- 0..1 medipointsentries (unique: one earn entry per order)
```

All references are ObjectIds with declared indexes. Order lines intentionally carry both the reference and a copy of the name, image, unit price and line total: the reference supports analytics and repeat purchase, while the copy preserves history (FR-2.6).

Figures 7 to 9 give the entity-relationship view of the SRS data model in three parts: shopping, cart and wishlist (Figure 7); orders, prescriptions and payments (Figure 8); and customer services and security (Figure 9). Entities are rounded boxes, attributes are ellipses and relationships are diamonds, with cardinalities on the connectors. Like the class model, they show the SRS target. In the implemented collections (Table 20) payments, cart lines and order lines are embedded in their parent documents, and the wishlist, prescription and refill-reminder entities are not yet persisted.

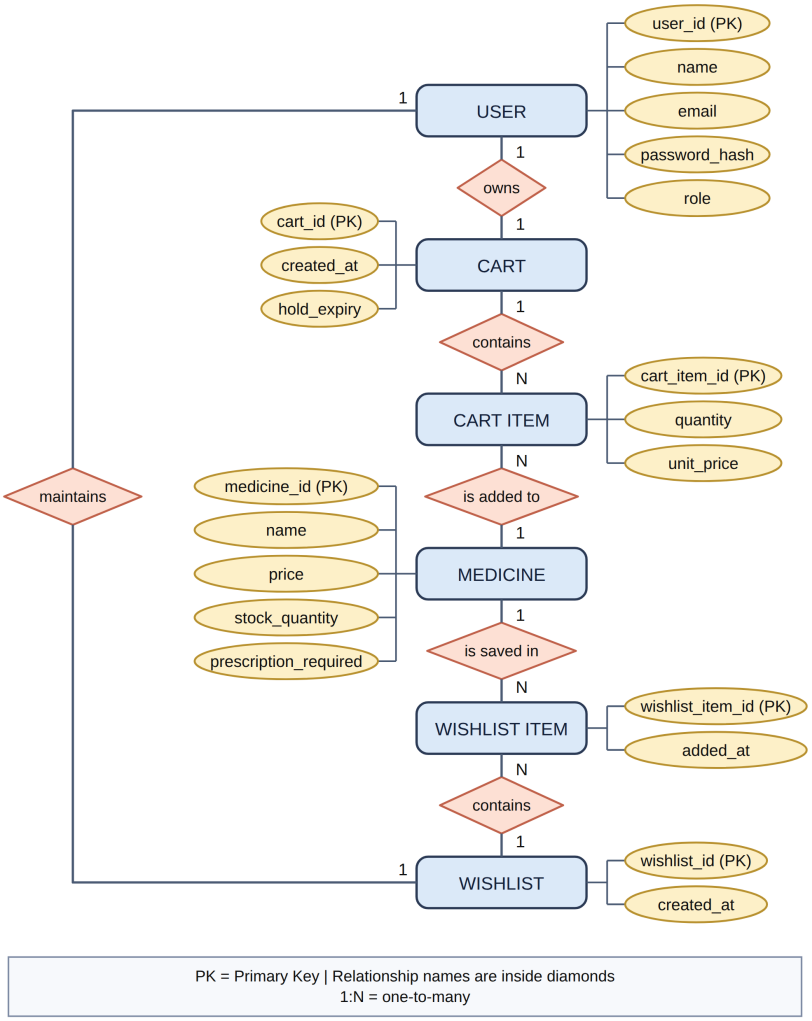


Fig. 7. ER diagram (part 1 of 3): shopping, cart and wishlist

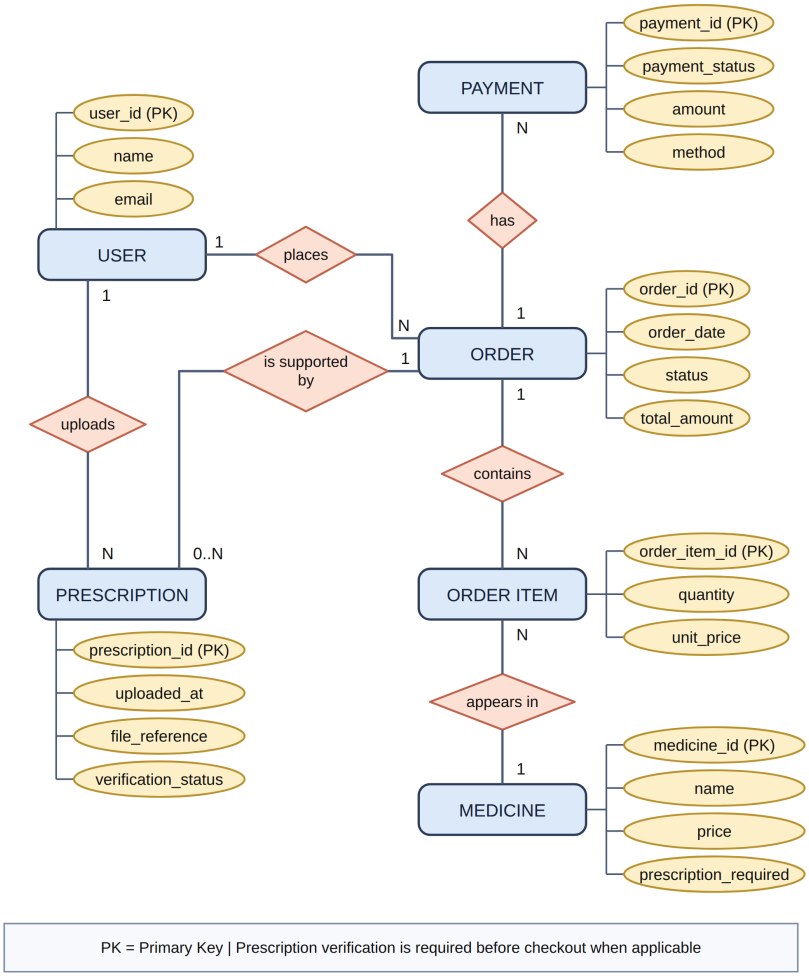


Fig. 8. ER diagram (part 2 of 3): orders, prescriptions and payments

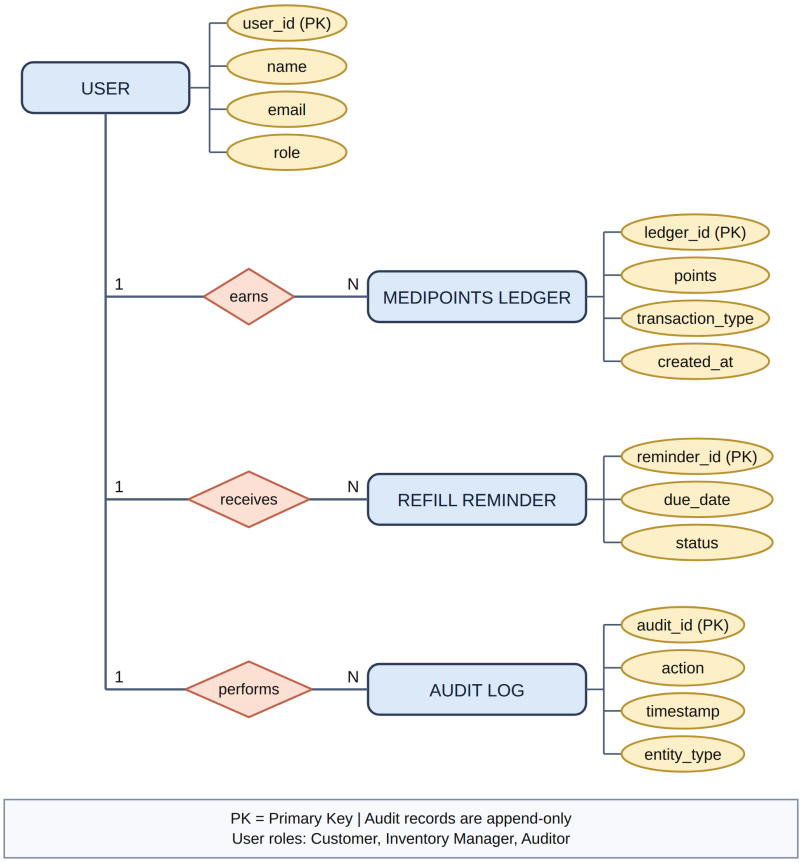


Fig. 9. ER diagram (part 3 of 3): customer services and security

6.3 API Specification

Table 22. API groups and representative endpoints

Group	Endpoints	Access
Auth	`POST /api/auth/register`, login, refresh, logout; `GET /api/auth/me`; `GET|DELETE /api/auth/sessions`; `DELETE /api/auth/sessions/:id`; `PATCH /api/auth/password`; `GET /api/auth/users`; `PATCH /api/auth/users/:id/role`	Public / signed in / admin
Back-office auth	`POST /api/admin/auth/login`; `POST /api/admin/auth/accept-invite`	Public; customers rejected
Back office	`GET /api/admin/summary`; `GET /api/admin/orders`, `/orders/:id`; `PATCH /api/admin/orders/:id/status`; `GET /api/admin/customers`; `POST /api/admin/inventory/replenish`; `GET /api/admin/audit`; `POST|GET /api/admin/invites`; `DELETE /api/admin/invites/:id`	staff / admin; audit and invites admin only
Stock	`GET /api/medicines`, `/medicines/:id`; `POST /api/medicines`; `PATCH /api/medicines/:id`; `DELETE /api/medicines/:id`	Public read; writes need `catalog:write`
Cart	`GET /api/cart`; `POST /api/cart/items`; `PATCH|DELETE /api/cart/items/:medicineId`; `DELETE /api/cart`; `POST /api/cart/merge`	Guest cookie or JWT
Orders	`POST /api/orders`; `GET /api/orders`, `/orders/:id`; `POST /api/orders/:id/pay`; `POST /api/orders/:id/cancel`	Owner
Account	`GET /api/me/medipoints`	Signed in
Platform	`GET /api/health`; `GET /uploads/:file`	Public

6.4 Error Conventions

Every failure returns JSON of the form `{ "error": "..." }`, optionally with additional context (a declined payment returns `{ error, order }`). Status codes are used consistently:

- 400 — malformed or invalid input (missing field, bad quantity, invalid refill cycle, unsupported payment method).
- 401 — missing, expired or revoked credentials; the SPA responds by refreshing once and retrying.
- 402 — payment declined by the provider; the order stays payable and the reason is stored on it.
- 403 — authenticated but not permitted (customer attempting a staff action, customer on the back-office login).
- 404 — unknown resource, including another user's order, which is deliberately not distinguished from a missing one.
- 409 — state conflict (duplicate e-mail, out of stock, expired reservation, illegal status transition).
- 415 — form-encoded body sent to a JSON-only endpoint.
- 429 — rate limit exceeded, returned as JSON rather than an HTML page.
- 500 / 502 — unexpected server error or provider unavailable; the body never exposes a stack trace.

7 Verification and Validation

This clause is organised in the sequence recommended by IEEE Std 829-2008: strategy, environment, test inventory, representative test cases, execution results and coverage. Every number in Clause 7.5 was produced by running the suite on the development machine in this revision; nothing is projected or estimated.

7.1 Verification Strategy

Verification is primarily dynamic and end-to-end. Each suite starts a real MongoDB in memory, connects the real database helper, starts the real Express application on an ephemeral port and issues real HTTP requests with `fetch`. No controller, router, model or database call is stubbed. This was a deliberate choice: the requirements that matter most here (reservation atomicity, single-shot stock restoration, idempotent payment, session rotation) live at the boundaries between layers, and unit tests with mocks would not observe them.

Table 23. Test levels applied

Level	Applied to	Consequence
Unit	Pure helpers: pricing, availability, connection-string hints, permission map, MediPoints rule, inventory configuration	Fast feedback with no I/O; includes negative cases such as an invalid refill target
Integration / API	Every route group exercised over HTTP with real Mongoose models and indexes	Catches authorisation gaps, validation drift and serialisation errors

Table 23 (continued)

Level	Applied to	Consequence
Concurrency	Parallel reservation and replenishment passes (`Promise.all`) and repeated transitions	Demonstrates that the conditional-update guards hold
Build / static	Frontend production build; `node --check` on every changed server file	Catches syntax and bundling errors before review
Manual exploratory	Scripted end-to-end run of the storefront flow against a live server	Confirms the timers, the UI-facing shapes and the README instructions

7.2 Test Environment

- **Runtime:** Node.js 22.22.3, npm 10.9.8, Linux (x86-64).
- **Database:** mongodb-memory-server 11 — an ephemeral real MongoDB per test file, so no installation or shared state is required.
- **Runner:** `node -test` (built-in), `node:assert/strict`; suites run in isolated processes.
- **Determinism:** the payment simulator latency is set to 0 ms in tests; the replenishment sweep is disabled in the inventory suite so assertions cannot race the timer; timestamps inside tests are relative to the current clock.
- **Isolation:** each suite uses its own database name, its own port (0 = ephemeral) and its own cookie jar, so suites can run in any order or in parallel.

7.3 Test Suite Inventory

Table 24. Test suites and scope

File	Cases	Scope
`api.test.js`	29	Health and security headers; registration and login validation; role-based stock writes; stock CRUD and image upload; validation failures; 404s; login rate limiting; catalogue ordering
`cart-orders.test.js`	21	Signed cart cookie; guest cart and merge; quantity, stock and expiry validation; server-side totals; checkout snapshot and reservation; rollback; card, UPI, net-banking and COD payments; declines and retries; cancellation; reservation expiry; ownership; order history
`inventory.test.js`	12	Restock at the watermark; rows above the watermark untouched; zero-stock restock; expired stock excluded; configurable watermark/target; concurrent sweeps restock once; RBAC on the endpoint; audit entry; dashboard configuration; Rx and refill fields with validation
`medipoints.test.js`	9	₹100-block earning rule and configurable rate; authentication required; zero starting balance; credit after a confirmed card payment; no double credit on replay; decline-then-retry credits once; COD earns nothing; per-customer isolation
`admin.test.js`	9	Role split (customer/staff/admin); staff stock rights; invite creation, acceptance, expiry and revocation; order fulfilment transitions; customer statistics; audit log contents and access
`sessions.test.js`	8	Refresh-token rotation; reuse detection revoking all sessions; logout; password change invalidating access tokens; listing and revoking another device; sign-out everywhere
`mongoErrors.test.js`	8	Connection-string redaction, local/cloud classification, targeted hints for bad auth, DNS/SRV failures and timeouts

Table 24 (continued)

File	Cases	Scope
`config.test.js`	1	Production refuses to start without JWT_SECRET and COOKIE_SECRET
Total	97	All passing (Clause 7.5)

7.4 Representative Test Cases

Table 25. Representative test cases with expected and observed results

ID	Requirement	Input / action	Expected	Observed
TC-01	FR-1.1	Medicine at stock 10; `POST /api/admin/inventory/replenish` as staff	200, one row restocked 10→100, stock now 100	Pass
TC-02	FR-1.1	Two concurrent replenishment passes on a row at stock 2	Exactly one pass reports the row; final stock 100 (not 200)	Pass
TC-03	FR-1.1	Expired medicine at stock 3; replenish	Row skipped; stock remains 3	Pass
TC-04	FR-1.1, NFR-1.1	Anonymous and customer callers on the replenish endpoint	401 and 403	Pass
TC-05	FR-2.3	Pay by net banking with bank `hdfc`	200, status paid, method netbanking, brand hdfc, no last4	Pass
TC-06	FR-2.3	Net banking with unknown bank, `failbank` and no bank; then retry with `sbi`	402 three times with distinct reasons; retry succeeds	Pass
TC-07	FR-2.2	Card 4000 0000 0000 0002 (Luhn-valid, forced decline), then UPI retry	402 with order still `pending_payment`; retry pays	Pass
TC-08	FR-2.4	₹250 subtotal paid by card	Balance 2 points, one ledger entry linked to the order	Pass
TC-09	FR-2.4	Replay the payment request on an already-paid order	200 without re-charging; balance unchanged	Pass
TC-10	FR-2.4	Cash-on-delivery order confirmed	Status processing, payment pending, no points	Pass
TC-11	NFR-2.1	Two buyers race for the last unit	One order created, the other 409; stock never negative	Pass
TC-12	NFR-2.1	Multi-line checkout where a later line is short	409 and every earlier reservation rolled back	Pass
TC-13	FR-2.6, NFR-2.1	Cancel an unpaid order, then cancel again	Stock restored once; second call a no-op	Pass
TC-14	FR-2.6	Let a reservation lapse, then read the order	Status expired, stock restored once	Pass
TC-15	NFR-1.2	Replay a rotated refresh token	All sessions revoked; access tokens rejected	Pass
TC-16	NFR-1.2	Change the password, then use the previous access token	401 immediately	Pass
TC-17	NFR-1.1	Customer attempts `POST /api/medicines`; staff attempts it	403 for the customer, 201 for staff	Pass
TC-18	NFR-1.3	Restock, then read `/api/admin/audit?action=medicine.replenish` as admin	Entry present with before/after stock; staff gets 403	Pass
TC-19	FR-1.6	Register with `role: "admin"`	403 — privileged roles are never self-service	Pass

Table 25 (continued)

ID	Requirement	Input / action	Expected	Observed
TC-20	NFR-1.1	Exceed the failed-login limit for one e-mail	429 with a JSON error	Pass

7.5 Execution Results

Table 26. Executed test results (this revision)

Suite	Cases	Passed	Failed	Notes
`tests/api.test.js`	29	29	0	Includes rate-limit and upload cases
`tests/cart-orders.test.js`	21	21	0	Four payment methods, declines, expiry
`tests/inventory.test.js`	12	12	0	Sweeper disabled in this suite for determinism
`tests/medipoints.test.js`	9	9	0	Includes idempotency and COD cases
`tests/admin.test.js`	9	9	0	Invites, fulfilment, audit
`tests/sessions.test.js`	8	8	0	Rotation and reuse detection
`tests/mongoErrors.test.js`	8	8	0	Pure unit tests, no database
`tests/config.test.js`	1	1	0	Production secret guard
Total	97	97	0	Full run 3.5–6.0 s wall clock (warm cache; a cold run measured 6.0 s); no skips, no todos

The suite is reproducible with `cd backend && npm test` and requires no MongoDB installation (Appendix E). The per-file counts above were obtained by running each suite individually as well as the full set, and the two agree.

7.6 Build and Static Verification

Table 27. Build verification

Check	Command	Observed result
Frontend production build	`cd frontend && npm run build`	Vite 5.4.21, 64 modules transformed; CSS 22.22 kB (gzip 4.79 kB); JS 242.36 kB (gzip 71.52 kB); built in ≈1–2 s
Server syntax	`node --check` on changed files	No syntax errors in 10 files
Backend start-up	`npm run dev` against a local MongoDB	Connects, applies migrations, runs the startup sweep, logs the listening port
Guidance script	`npm run db:check`	Prints the redacted target, database name, server version, collections and document counts

7.7 Requirement-to-Test Coverage

Table 28. Requirement coverage by automated tests

Requirement	Covering suite(s)	Coverage judgement
FR-1.1	inventory.test.js	Direct: watermark, target, expiry exclusion, configuration, concurrency, RBAC, audit
FR-1.2, FR-1.3	cart-orders.test.js	Order-level reservation, rollback, expiry and single-shot release covered; cart-level holds do not exist yet
FR-1.4, FR-2.1	inventory.test.js (field only)	The Rx flag round-trips through the API; the verification gate is not implemented, so it cannot be tested

Table 28 (continued)

Requirement	Covering suite(s)	Coverage judgement
FR-1.5	—	Not covered (feature planned)
FR-1.6, FR-1.7	api.test.js, sessions.test.js	Direct: validation, role forcing, login, refresh, revocation
FR-2.2	cart-orders.test.js	Direct: decline, retry, rollback, no partial decrement
FR-2.3	cart-orders.test.js	Direct: card success/decline, UPI success/decline, net banking success/declines, COD limit
FR-2.4	medipoints.test.js	Direct: rule, gating on payment state, idempotency, isolation
FR-2.5	inventory.test.js (field only)	Refill cycle is stored; reminder workflow planned
FR-2.6	cart-orders.test.js, admin.test.js	Snapshot values asserted at checkout; status transitions restricted by a fixed map
NFR-1.1	api.test.js, admin.test.js, inventory.test.js	401/403 across stock, back office, audit and replenishment
NFR-1.2	api.test.js, sessions.test.js, config.test.js	Hashing, token expiry, rotation, reuse detection, production secret guard
NFR-1.3	admin.test.js, inventory.test.js	Writes and reads covered; the absence of an update/delete route is verified by inspection, not by a test asserting the route does not exist
NFR-1.4	cart-orders.test.js	Brand/last4-only persistence is asserted; prescription encryption pending
NFR-2.1	cart-orders.test.js, inventory.test.js	Concurrency and rollback both exercised
NFR-2.2	—	UI countdown verified manually only; no browser automation
NFR-2.3	—	Goal; no uptime test possible in a single-host prototype

7.8 Threats to Validity

- **Internal validity.** Tests run against mongodb-memory-server, a real but ephemeral MongoDB. Behaviour that depends on a replica set (multi-document transactions), on network latency between application and database, or on Atlas-specific settings is therefore not exercised.
- **Concurrency realism.** Concurrency is simulated with `Promise.all` inside one process. This exercises the conditional updates but not true multi-process contention; a load generator would strengthen the claim.
- **External validity.** No load, soak, accessibility or cross-browser testing was performed. The frontend is verified by a successful production build and a manual walkthrough, not by browser automation.
- **Construct validity.** Requirement wording is interpreted, and Table 8 records every such interpretation so a reviewer can disagree explicitly rather than discover a silent redefinition.
- **Conclusion validity.** A passing suite shows that asserted behaviour holds; it does not prove the absence of defects, and several requirements (FR-1.5, FR-2.1, FR-2.5, NFR-2.2, NFR-2.3) have no automated coverage.

8 Results and Discussion

8.1 Summary of Results

The prototype delivers a coherent end-to-end workflow — browse, cart, checkout, pay, fulfil — with the consistency and access-control properties the SRS treats as high priority, and it does so under the constraints of a three-person, one-semester project. Table 29 collects the measured

indicators. Only quantities that were observed are listed; no throughput or latency figure is claimed, because no load test was performed.

Table 29. Measured project indicators

Indicator	Measured value	Basis
Automated test cases	97	Eight suites, `node -test`
Pass rate	97 / 97 (100 %)	Full-suite runs in this revision
Full-suite wall-clock time	3.5–6.0 s	Four runs, warm and cold cache
Backend source (non-test)	≈ 4,713 lines	models + routes + lib + middleware + scripts
Backend tests	2,551 lines	≈ 35 % of backend code
Frontend source	≈ 6,284 lines	Pages, components, contexts, helpers, stylesheet
REST route handlers	41	40 router handlers plus `/api/health`
Mongo collections / models	8 (+1 internal)	Table 20
Frontend production bundle	242.36 kB JS (71.52 kB gzip), 22.22 kB CSS	`vite build`
Functional requirements	7 verified, 2 partial, 4 planned	Table 6, Appendix A
Non-functional requirements	4 verified, 2 partial, 1 goal	Table 7

8.2 Requirements Alignment

Table 30. Requirement-by-requirement assessment

SRS area	Assessment	Evidence
Live stock catalogue (≈50 items)	Implemented	Public listing with live stock, price, dates and images; seeding to approximately 50 items is a demo task (Phase F)
Replenishment at the watermark	Implemented	Sweep plus manual endpoint, configuration-driven, expired rows excluded, audited (TC-01 to TC-04)
Cart soft hold and automatic release	Partial	Atomic reservation exists at order creation; cart-level holds and their sweeper are Phase C
Prescription detection and billing block	Planned	The Rx flag exists and is staff-editable; upload, verification queue and checkout gate are Phase D
Wishlist	Planned	Phase B
Customer registration and login	Implemented	Customer-only registration, JWT access tokens, rotating refresh sessions (TC-15, TC-16, TC-19)
Payment methods (UPI, debit, credit, net banking)	Implemented	Four methods through the sandbox adapter (TC-05, TC-06)
Payment failure leaves state unchanged	Implemented	The decline path updates payment fields only; retry succeeds (TC-07)
MediPoints after confirmed payment	Implemented	Ledger with a unique index, verified for card, replay and COD (TC-08 to TC-10)
Refill reminders to teleconsultation	Planned	Refill cycle stored; reminder and teleconsultation request are Phase E
Immutable purchase history	Implemented	Order snapshots, no mutating routes, restricted transitions (TC-13, TC-14)
RBAC on every protected request	Implemented	Permission checks per route (TC-17, TC-20)
bcrypt + JWT with expiry	Implemented	Ten-round hashing, 15-minute tokens, 30-day refresh sessions, revocation (TC-15, TC-16)
Append-only audit log	Implemented	Audited writes, read-only listing, no update/delete route (TC-18)
Payment data encrypted or tokenised at rest	Partial	No card data retained; prescription encryption is Phase D
Atomic reservation	Implemented	Conditional updates with rollback (TC-11, TC-12)
Hold time and Rx status at checkout	Partial	Reservation countdown on the payment page; cart countdown and Rx status arrive with Phases C and D
24×7 operational goal	Goal	Stateless API and idempotent operations; no monitoring, backup or SLA evidence

Table 30 (continued)

SRS area	Assessment	Evidence
QR/barcode simulation	Planned	Phase F; no hardware dependency exists

8.3 Discussion of Trade-offs

Correctness without transactions. The most consequential decision was to make every stock and status change a single-document conditional update. It keeps the prototype portable to a standalone MongoDB and gives one clear invariant per operation, but it moves atomicity for a multi-line checkout into application logic: the code must remember to compensate earlier lines, and that compensation is exactly what the rollback branch does. A replica set with a transaction would remove the compensation code at the cost of a heavier development environment.

Where holds belong. Reserving at checkout (current behaviour) keeps the cart simple and makes the reservation window coincide with the payment window, when a customer is most likely to convert. The SRS asks for a hold at add-to-cart, which protects a customer who is still shopping but needs a second stock counter (`held`), a release sweeper and a conversion step at checkout. Phase C implements the SRS reading and must keep both counters consistent; that is the highest-risk change left in the plan and deserves its own suite before any UI work.

Replenishment as a background event. Restocking inside the checkout path would have simplified the demo but would make a sale mutate stock in two opposite directions within one request, weakening both the audit story and the reservation arithmetic. A separate sweep keeps a sale honest and models a supplier delivery, at the cost of stock remaining at the watermark until the next pass — visible in the demo by design and controllable through the manual endpoint.

A ledger instead of a counter. Storing a MediPoints balance on the user would have been one field and one update. The ledger costs an aggregation per read but buys an auditable history and a database-enforced "once per order" rule that survives a replayed request — precisely the failure mode FR-2.4 is about.

8.4 Limitations

- **Unfinished SRS scope.** Cart soft holds, prescription gating, wishlist, refill reminders and QR/barcode simulation are specified but not built (Tables 6 and 30).
- **No batch-level inventory.** One stock counter and one expiry date per medicine; several batches of the same medicine with different expiry dates cannot be represented. The repository README records the embedded-batch shape to adopt if that becomes a requirement.
- **Single process.** Rate limiting and the replenishment sweep assume one API instance; scaling out needs a shared limiter store and an elected sweeper.
- **Local file storage.** Medicine images live on the API host's disk, so a multi-host deployment needs object storage.
- **No performance evidence.** No load, soak or profiling work was done; this report claims functional correctness, not performance.
- **Limited frontend verification.** Production build plus manual walkthrough; no unit tests, browser automation or accessibility audit for the React code.
- **Prototype operations.** No monitoring, backup or recovery procedure, and TLS termination is delegated to the hosting platform.

8.5 Risks and Mitigations

Table 31. Risk register

Risk	Likelihood	Impact	Mitigation
Cart-hold work (Phase C) breaks reservation accounting and causes overselling	Medium	High	Introduce the `held` counter behind its own test suite; assert stock + held invariants; convert holds to sales in one conditional update
Requirement drift between SRS revisions	Medium	Medium	Appendix A is the single traceability source; any change is recorded in Table 8 before code is written
Prescription upload exposes patient data	Medium	High	Phase D: AES-256-GCM at rest, storage outside the static directory, access limited to the owner and a verifier, access logged
Privilege escalation through a missed authorisation check	Low	High	Permission middleware on every protected route; 401/403 tests per group; role re-read on each request
Secret leakage from development configuration	Low	High	No secrets committed; production refuses to boot without secrets; connection strings redacted in logs
Concurrency defects missed by in-process tests	Medium	Medium	Add a multi-process load test against the reservation endpoint before claiming production readiness
Team availability in the final weeks	Medium	Medium	Phases are independently shippable and each ends with tests and documentation, so the project stays presentable at any point

9 Conclusion and Future Work

9.1 Conclusion

MediTrack is a working three-tier pharmacy storefront with a back-office console, built against a formal SRS within the constraints of a one-semester, three-person project. The parts of the SRS that carry the highest risk — inventory consistency, payment failure handling, privilege separation, session security and auditability — are implemented and are covered by 97 passing automated tests. The parts that remain are clearly identified rather than hidden: cart-level soft holds, prescription verification, a wishlist, refill reminders and QR simulation.

Three outcomes are worth carrying forward. First, the prototype demonstrates that the important invariants of an e-commerce checkout can be defended without multi-document transactions, by expressing each one as a conditional single-document update with an explicit compensation path. Second, the project shows how a requirement can be traced to a file and to an executing test, which is what makes the status claims in this report checkable rather than rhetorical. Third, the remaining work is not a research problem but a sequencing problem: each outstanding requirement has a defined data model, endpoint and test plan, and the phases are ordered so that each one leaves the system in a demonstrable state.

9.2 Future Work

Remaining work is organised as five increments. Each increment is independently reviewable, adds its own tests, and updates Appendix A so the traceability matrix never lags behind the code.

Table 32. Future work mapped to requirements

Phase	Scope	Requirements closed	Design notes
B	Wishlist: Wishlist collection, `/api/wishlist` routes, move-to-cart, storefront page, navigation badge	FR-1.5	One wishlist per user; move-to-cart reuses `getOrCreateCart` and the existing stock checks
C	Cart-level soft holds: `Medicine.held` counter, `holdExpiresAt` per cart line, release sweeper, hold-to-sale conversion, cart countdown	FR-1.2, FR-1.3, NFR-2.2	Guard every add with `$expr: stock − held ≥ qty`; convert at checkout with `$inc {stock: −qty, held: −qty}`; never let the two counters drift
D	Prescriptions: upload (PDF/JPG/PNG), AES-256-GCM at rest, verification queue, checkout gate, Rx status at checkout and on the receipt	FR-1.4, FR-2.1, NFR-1.4, NFR-2.2	One verified prescription per customer valid for a configurable period; files stored outside the static directory and streamed only to the owner or a verifier
E	Refill reminders from purchase history and a teleconsultation request	FR-2.5	Computed from last purchase + `refillDays`; the action books an assisted consultation instead of re-dispatching medicine (ASSUME-1)
F	Hardening and demo readiness: schema-level immutability guards, simulated QR token with an admin scan endpoint, catalogue seeding to ≈50 medicines, README and traceability refresh	FR-2.6 (defence in depth), NFR-1.3, SRS 4.5	No new dependency for the QR simulation; guards written as Mongoose middleware so a script cannot bypass them

Operational work beyond the prototype. Before any real deployment the team should add monitoring and alerting, scheduled backups with a restore drill, a shared rate-limit store and a single elected sweeper for multi-instance hosting, object storage for uploads, a content-security policy for the served SPA, dependency scanning in CI, and a load test that exercises the reservation path from several processes. These are recorded here because the SRS explicitly excludes production guarantees (NFR-2.3) while the architecture should not make them impossible.

References

[1] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, Institute of Electrical and Electronics Engineers, 1998.

[2] MediTrack Development Team, *MediTrack — Software Requirements Specification, Version 1.0*, 13 pp., prepared per IEEE Std 830 structure, 2026.

[3] IEEE, *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*, IEEE Std 1016-2009, Institute of Electrical and Electronics Engineers, 2009.

[4] IEEE, *IEEE Standard for Software and System Test Documentation*, IEEE Std 829-2008, Institute of Electrical and Electronics Engineers, 2008.

[5] E. Alba, *Writing a Scientific/Technical Report*, Departamento de Lenguajes y Ciencias de la Computación, Universidad de Málaga, 2002.

[6] MediTrack Project Team, *MediTrack source repository and README* (commit `ca672d9` plus the Phase A working tree), 2026.

[7] MongoDB Inc., *MongoDB Manual: indexing, unique and partial indexes, TTL indexes, atomic updates*, 2026. [Online]. Available: `https://www.mongodb.com/docs/manual/`

[8] OpenJS Foundation, *Node.js Documentation: test runner, crypto, http*, 2026. [Online]. Available: `https://nodejs.org/api/`

[9] Express.js, *Express 4.x API Reference* and Mongoose, *Mongoose 7 Documentation*, 2026. [Online]. Available: `https://expressjs.com/` and `https://mongoosejs.com/docs/`

[10] Meta Open Source, *React Documentation* and Vite, *Vite Guide*, 2026. [Online]. Available: `https://react.dev/` and `https://vitejs.dev/guide/`

Appendix A. Requirements Traceability Matrix

Each row maps one SRS requirement to the artefact that implements it and to the evidence that verifies it. "Verified" means an automated test asserts the behaviour; "partial" means the mechanism exists but does not yet meet the SRS wording; "planned" means the requirement is scheduled in Table 32 and has no implementation to point at.

Table 33. Traceability from requirement to implementation and evidence

ID	Prio.	Requirement (abridged)	Status	Implementation	Evidence
FR-1.1	High	Replenish at watermark 10 → 100	Verified	`lib/inventory.js`; `server.js` sweep; `POST /api/admin/inventory/replenish`; `scripts/replenish-inventory.js`	`tests/inventory.test.js` (12)
FR-1.2	High	Atomic soft hold on add-to-cart	Partial	`lib/orderService.js` (reservation at checkout); Phase C adds `held`	`tests/cart-orders.test.js`
FR-1.3	High	Release expired hold automatically	Partial	`expireStaleOrders`, `expireOrderById`	`tests/cart-orders.test.js`
FR-1.4	High	Detect Rx-only items and block billing	Planned	`Medicine.prescriptionRequired`; gate in Phase D	Field asserted in `tests/inventory.test.js`
FR-1.5	Medium	Wishlist independent of cart	Planned	Phase B	—
FR-1.6	High	Customer account creation	Verified	`routes/auth.js`	`tests/api.test.js`
FR-1.7	High	Customer login	Verified	`routes/auth.js`, `lib/sessions.js`	`tests/api.test.js`, `tests/sessions.test.js`
FR-2.1	High	Block checkout until Rx verified	Planned	Phase D	—
FR-2.2	High	Failed payment leaves state unchanged	Verified	`lib/orderService.js`, `routes/orders.js`	`tests/cart-orders.test.js`
FR-2.3	High	UPI, debit, credit, net banking (sandbox)	Verified	`lib/providers/mockPayment.js`	`tests/cart-orders.test.js`
FR-2.4	Medium	MediPoints after confirmed payment	Verified	`lib/medipoints.js`, `models/MediPointsEntry.js`, `routes/me.js`	`tests/medipoints.test.js` (9)
FR-2.5	Medium	Refill reminder → teleconsultation	Planned	`Medicine.refillDays`; workflow in Phase E	Field asserted in `tests/inventory.test.js`
FR-2.6	High	Immutable purchase history	Verified	`models/Order.js` snapshots; no mutating route	`tests/cart-orders.test.js`, `tests/admin.test.js`
NFR-1.1	High	RBAC on every protected request	Verified	`middleware/auth.js`, `lib/permissions.js`	`tests/api.test.js`, `tests/admin.test.js`, `tests/inventory.test.js`
NFR-1.2	High	bcrypt + JWT with session expiry	Verified	`models/User.js`, `models/Session.js`, `lib/sessions.js`	`tests/sessions.test.js`, `tests/config.test.js`
NFR-1.3	High	Append-only audit log	Verified	`lib/audit.js`, `models/AuditLog.js`	`tests/admin.test.js`, `tests/inventory.test.js`
NFR-1.4	High	Tokens and prescription data protected	Partial	No card data stored; prescription encryption in Phase D	`tests/cart-orders.test.js` (brand/last4 only)

Table 33 (continued)

ID	Prio.	Requirement (abridged)	Status	Implementation	Evidence
NFR-2.1	High	Atomic reservation, no overselling	Verified	`lib/orderService.js`	`tests/cart-orders.test.js`
NFR-2.2	Medium	Show hold time and Rx status	Partial	`pages/Payment.jsx` countdown; Phase C/D complete it	Manual walkthrough
NFR-2.3	Low	24×7 operational goal	Goal	Stateless API, idempotent operations, graceful shutdown	Not testable in this prototype

Appendix B. API Reference

Table 34. API surface (part 1: platform, identity and back office)

Method	Path	Access	Purpose
GET	`/api/health`	Public	Status and database connection state
POST	`/api/auth/register`	Public, rate limited	Create a customer account (privileged roles rejected)
POST	`/api/auth/login`	Public, rate limited	Issue an access token and a refresh cookie
POST	`/api/auth/refresh`	Refresh cookie	Rotate the session and return a new access token
POST	`/api/auth/logout`	Refresh cookie	Revoke this browser's session
GET	`/api/auth/me`	Signed in	Current user and permissions
GET	`/api/auth/sessions`	Signed in	List active sessions, marking the current one
DELETE	`/api/auth/sessions/:id`	Signed in	Revoke one session
DELETE	`/api/auth/sessions`	Signed in	Revoke every session and invalidate access tokens
PATCH	`/api/auth/password`	Signed in	Change the password and revoke all sessions
GET	`/api/auth/users`	`users:manage`	List accounts and roles
PATCH	`/api/auth/users/:id/role`	`users:manage`	Promote or demote an account (last-admin guard)
POST	`/api/admin/auth/login`	Public	Back-office sign-in; customers rejected
POST	`/api/admin/auth/accept-invite`	Public + invite token	Create the invited staff/admin account
GET	`/api/admin/summary`	`orders:read:all`	Counters, revenue, low-stock count and inventory configuration
GET	`/api/admin/orders`	`orders:read:all`	All orders with status filter, search and paging
GET	`/api/admin/orders/:id`	`orders:read:all`	One order with the customer
PATCH	`/api/admin/orders/:id/status`	`orders:write`	Fulfilment transition from the allowed map
GET	`/api/admin/customers`	`customers:read`	Customers with order count, spend and last order
POST	`/api/admin/inventory/replenish`	`catalog:write`	Restock everything at or below the watermark
GET	`/api/admin/audit`	`audit:read`	Paginated audit log, filterable by action
POST	`/api/admin/invites`	`invites:manage`	Create a single-use invite and return its link once
GET	`/api/admin/invites`	`invites:manage`	List pending invites (optionally closed ones)
DELETE	`/api/admin/invites/:id`	`invites:manage`	Revoke a pending invite

Table 34 (continued). API surface (part 2: stock, cart, orders and account)

Method	Path	Access	Purpose
GET	`/api/medicines`	Public	List medicines (newest first)
GET	`/api/medicines/:id`	Public	Read one medicine
POST	`/api/medicines`	`catalog:write`	Create a medicine (multipart, optional image)
PATCH	`/api/medicines/:id`	`catalog:write`	Update fields, stock, Rx flag, refill cycle or image
DELETE	`/api/medicines/:id`	`catalog:write`	Remove a medicine and its image
GET	`/api/cart`	Guest cookie / JWT	Cart with live prices, stock, issues and totals

Table 34 (continued)

Method	Path	Access	Purpose
POST	`/api/cart/items`	Guest cookie / JWT	Add or increment a line
PATCH	`/api/cart/items/:medicineId`	Guest cookie / JWT	Set an exact quantity (0 removes)
DELETE	`/api/cart/items/:medicineId`	Guest cookie / JWT	Remove a line
DELETE	`/api/cart`	Guest cookie / JWT	Empty the cart
POST	`/api/cart/merge`	JWT	Merge the guest cart into the account cart
POST	`/api/orders`	JWT	Create an order from the cart and reserve stock
GET	`/api/orders`	JWT	Own order history (newest first)
GET	`/api/orders/:id`	JWT (owner)	One order; foreign ids return 404
POST	`/api/orders/:id/pay`	JWT (owner)	Charge through the provider; idempotent; credits points once
POST	`/api/orders/:id/cancel`	JWT (owner)	Cancel an unpaid order and release stock
GET	`/api/me/medipoints`	JWT	MediPoints balance, lifetime total and recent ledger entries
GET	`/uploads/:file`	Public	Serve an uploaded medicine image

Payment request bodies. `POST /api/orders/:id/pay` accepts `{ method: "card", details: { number, expiry, cvv } }`, `{ method: "upi", details: { vpa } }`, `{ method: "netbanking", details: { bank } }` and `{ method: "cod" }`. A decline returns 402 with `{ error, order }`; an unknown method returns 400; an expired or non-payable order returns 409.

Appendix C. Configuration Reference

All settings are read from `backend/.env`, which is never committed; `.env.example` documents every key. In production the server refuses to start without `JWT_SECRET` and `COOKIE_SECRET`.

Table 35. Configuration variables

Variable	Default	Effect
`PORT`	5000	HTTP port
`MONGODB_URI`	`mongodb://127.0.0.1:27017/meditrack`	Local or Atlas connection string
`MONGODB_DB`	`meditrack`	Database name used when the URI omits one
`JWT_SECRET`	— (dev fallback, warned)	Signing key for access tokens; mandatory in production
`JWT_ACCESS_TTL`	15m	Access-token lifetime (NFR-1.2)
`REFRESH_TOKEN_TTL_DAYS`	30	Refresh-session lifetime
`REFRESH_REUSE_GRACE_MS`	10000	Window in which a repeated rotation counts as a two-tab race, not theft
`COOKIE_SECRET`	— (dev fallback, warned)	Signs the cart and refresh cookies; mandatory in production
`ADMIN_INVITE_TTL_HOURS`	72	Lifetime of a staff/admin invite
`AUTH_RATE_LIMIT_WINDOW_MS`	900000	Window for authentication rate limits
`AUTH_LOGIN_RATE_LIMIT_MAX`	10	Failed sign-ins per IP + e-mail per window
`AUTH_REGISTER_RATE_LIMIT_MAX`	30	Registrations per IP per window
`CORS_ORIGIN`	(same origin)	Comma-separated allowed origins for cross-origin deployments
`TAX_RATE`	0.05	Tax applied to the subtotal
`DELIVERY_FEE`	40	Delivery charge below the threshold
`FREE_DELIVERY_OVER`	499	Subtotal at which delivery becomes free
`PAYMENT_PROVIDER`	`mock`	Provider registered in `lib/paymentGateway.js`
`PAYMENT_MOCK_DELAY_MS`	700	Simulated gateway latency (0 in tests)
`ORDER_RESERVATION_MINUTES`	30	How long an unpaid order holds its stock
`LOW_STOCK_WATERMARK`	10	FR-1.1 threshold that triggers replenishment
`REPLENISH_TO`	100	FR-1.1 target quantity after replenishment
`REPLENISH_INTERVAL_MS`	60000	How often the sweep looks for low stock
`INVENTORY_AUTO_REPLENISH`	`on`	Set to `off` to disable the sweep; the manual endpoint keeps working
`MEDIPOINTS_PER_100`	1	Points per whole ₹100 of medicine subtotal; 0 disables earning

Appendix D. Test Execution Log

The following excerpts are unedited console output from this revision. They are included so that the reported results can be checked against the artefacts.

Excerpt D.1 — full backend suite

```
$ cd backend && npm test
...
# Subtest: a medicine at the watermark is restocked to the target level
ok 1 - a medicine at the watermark is restocked to the target level
...
# Subtest: net banking succeeds through a demo bank and stores only the bank code
ok - net banking succeeds through a demo bank and stores only the bank code
...
1..97
# tests 97
# suites 0
# pass 97
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 3791.234159
```

Excerpt D.2 — per-suite counts

```
tests/admin.test.js          # pass 9  # fail 0
tests/api.test.js            # pass 29 # fail 0
tests/cart-orders.test.js    # pass 21 # fail 0
tests/config.test.js         # pass 1  # fail 0
tests/inventory.test.js      # pass 12 # fail 0
tests/medipoints.test.js     # pass 9  # fail 0
tests/mongoErrors.test.js    # pass 8  # fail 0
tests/sessions.test.js       # pass 8  # fail 0
```

Excerpt D.3 — frontend production build

```
$ cd frontend && npm run build
> meditrack-frontend@1.0.0 build
> vite build
vite v5.4.21 building for production...
✓ 64 modules transformed.
dist/index.html                   0.41 kB | gzip:  0.28 kB
dist/assets/index-C0tg3G4V.css   22.22 kB | gzip:  4.79 kB
dist/assets/index-BghP2PHm.js   242.36 kB | gzip: 71.52 kB
✓ built in 2.02s
```

Excerpt D.4 — manual end-to-end run (server with the sweep enabled)

```
[inventory] restocked 1: Smoke Rx 250 10->100
stock after sweep: 100 (expected 100)
cart add: 201 { total: 302.5 }
order: 201 { id: '6ae4...', subtotal: 250 }
netbanking pay: 200 { status: 'paid', method: 'netbanking', brand: 'hdfc' }
medipoints: 200 { balance: 2, entries: 1 }
summary inventory config: { lowStockWatermark: 10, replenishTo: 100 } lowStock: 0
```

Appendix E. Reproduction Procedure

E.1 Prerequisites

- Node.js 18 or later (verified on 22.22.3) with npm; Git; a modern browser.
- MongoDB 6 or later, either a local server or an Atlas cluster. No installation is needed to run the tests, because they start an in-memory MongoDB.

E.2 Backend

```
cd backend
npm install
cp .env.example .env          # then set MONGODB_URI, JWT_SECRET, COOKIE_SECRET
npm run db:check              # optional: verify the connection string before starting
ADMIN_EMAIL=admin@example.com ADMIN_PASSWORD='a-long-password' \
  ADMIN_NAME='Store Admin' npm run seed:admin
npm run dev                   # http://localhost:5000
```

If no MongoDB is available, run `npm run db:local` in one terminal (it uses the `mongod` binary downloaded by mongodb-memory-server and keeps data in `backend/.mongo-data`) and start the API from another terminal.

E.3 Frontend

```
cd frontend
npm install
npm run dev                   # http://localhost:5173, proxies /api and /uploads to :5000
```

E.4 Automated verification

```
cd backend && npm test        # expect: # pass 97, # fail 0
cd frontend && npm run build  # expect: ✓ built, no warnings
```

E.5 Demonstration script

1. Sign in at `/admin/login` as the seeded administrator, then open Stock → Add medicine and add a medicine with stock 10, a price of ₹250, the Rx box ticked and a refill cycle of 21 days.
2. Wait up to `REPLENISH_INTERVAL_MS` (or press Restock low stock): the quantity moves to 100 and an audit entry `medicine.replenish` appears under Audit.
3. Register a customer at `/login`, add the medicine to the cart and check out. The payment page shows the reservation countdown.
4. Pay by Net banking → HDFC. The order becomes Paid; the receipt shows the MediPoints earned (2 for a ₹250 subtotal) and the running balance.
5. Repeat with the demo bank that always declines to show the retry path, and with cash on delivery to show that points are withheld until the money is collected.

E.6 Verification checklist for a reviewer

- Every requirement in Appendix A has a status and, where verified, a named test file.
- `npm test` in `backend` reproduces 97 passing cases with no database installation.

- The three partial requirements (FR-1.2, FR-1.3, NFR-2.2) and the four planned ones (FR-1.4, FR-1.5, FR-2.1, FR-2.5) are the only gaps, and Table 32 dates them to phases B–F.
- No secret, token or card number appears in the repository, in the audit log or in an API response.

— End of report —